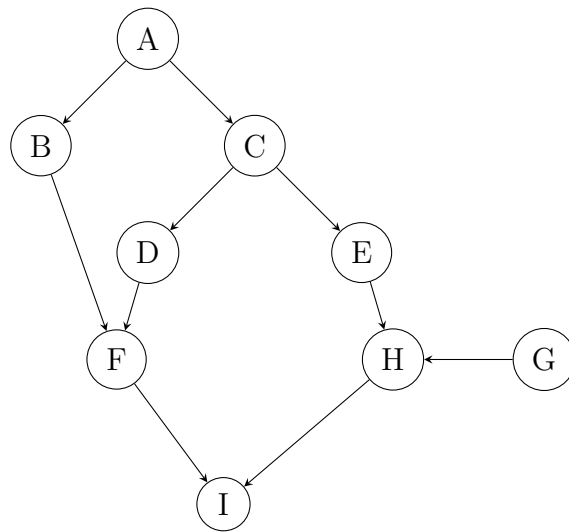


Coursework

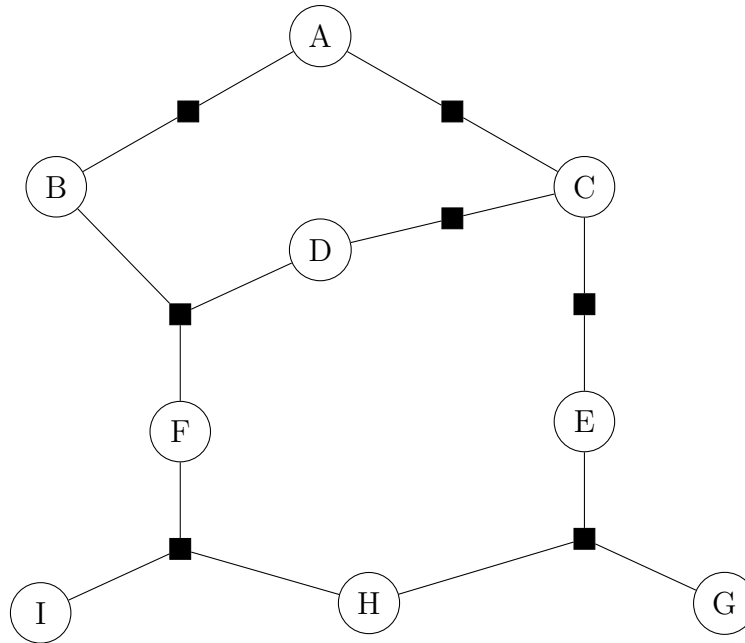
Due Date: 06/01/2025

Q1**(a)**

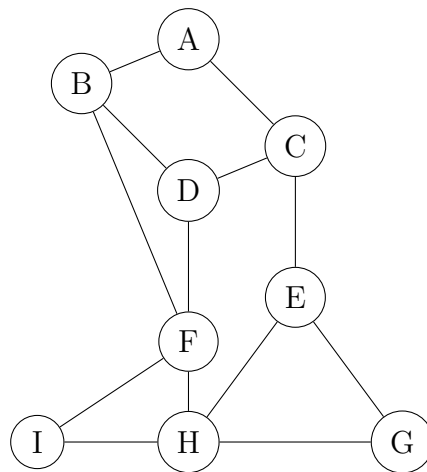
We draw the following DAG on the basis of the description.

**(b)**

Moralised Graph We first transform the DAG into a factored graph.



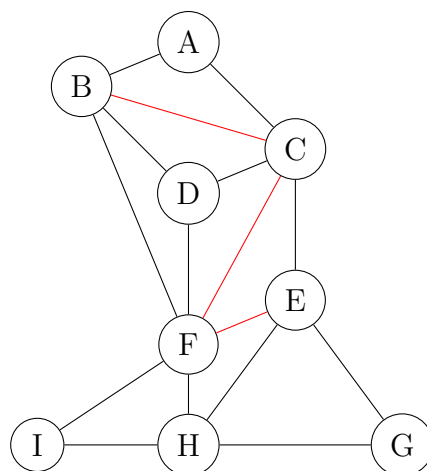
Then we moralise it to be an undirected graph.



Efficient Triangulation We perform the following order of variable elimination.

- Eliminate G and I, no edges added
- Eliminate H and draw EF
- Eliminate E and draw CF
- Eliminate F and draw BC
- Eliminate D, A, B and C in order and no edges added

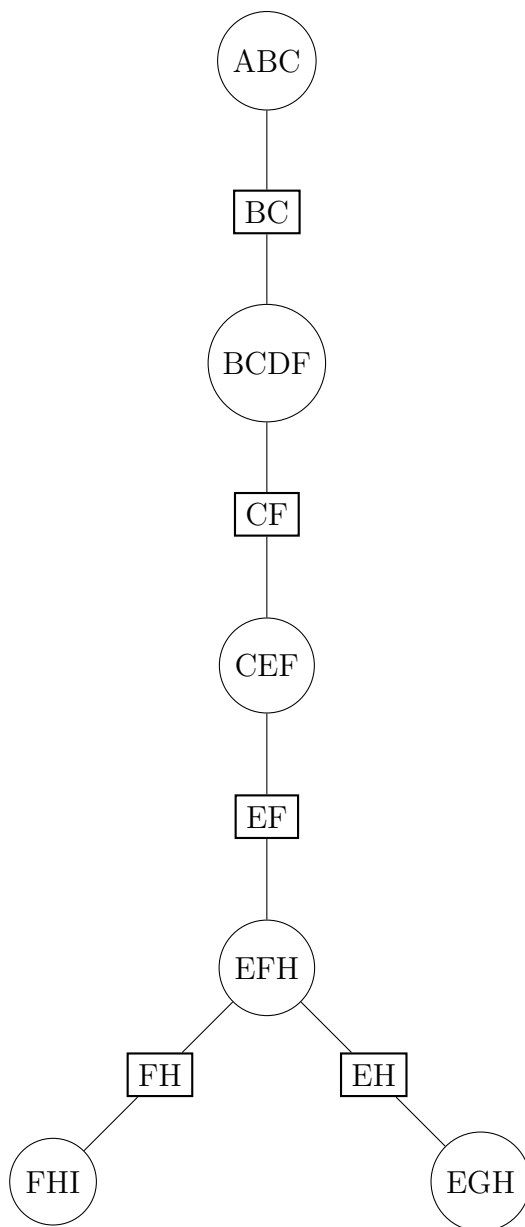
This is the graph after triangulation.



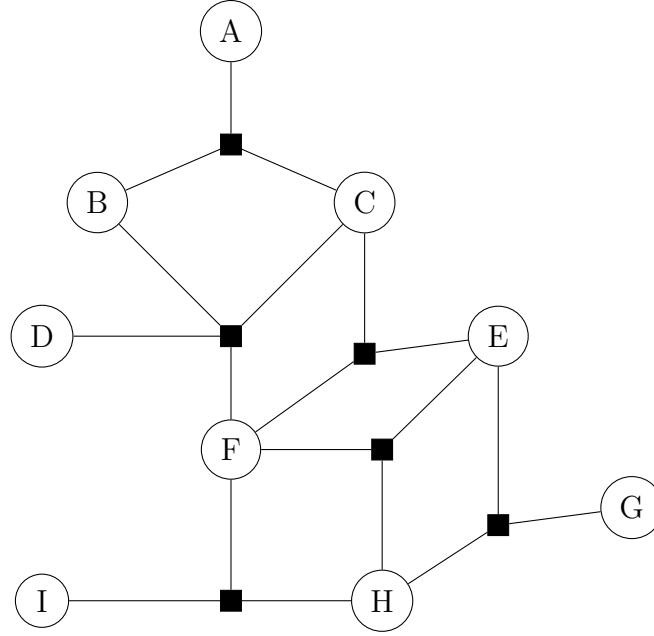
Resulting Junction Tree We have maximum cliques:

- $\{A, B, C\}$
- $\{B, C, D, F\}$
- $\{C, E, F\}$
- $\{E, F, H\}$
- $\{E, G, H\}$
- $\{F, H, I\}$

We put cliques in nodes and chords in edges, removing the edges where there is only 1 shared variable.



Junction Tree Redrawn as Factor Graph



(c)

We can start from the Markov Boundary of each node:

- $\mathcal{MB}(A) = \{B, C\}$
- $\mathcal{MB}(B) = \{A, D, F\}$
- $\mathcal{MB}(C) = \{A, D, E\}$
- $\mathcal{MB}(D) = \{B, C, F\}$
- $\mathcal{MB}(E) = \{C, G, H\}$
- $\mathcal{MB}(F) = \{B, D, H, I\}$
- $\mathcal{MB}(G) = \{E, H\}$
- $\mathcal{MB}(H) = \{E, F, G, I\}$
- $\mathcal{MB}(I) = \{F, H\}$

We need to find a set S such that $\mathcal{MB}(S) = \Omega \setminus S$ where Ω is the set of all variables.

By inspection, we can start from $S = \{B, C, G, I\}$, which meets the requirement. We can try to pop other elements from $M := \Omega \setminus S$:

- Pop A will require $B, C \in M$, shrinking the size of S
- Pop D will require $B, C \in M$, shrinking the size of S
- Pop E will require $C, G \in M$, shrinking the size of S

- Pop F will require $B, I \in M$, shrinking the size of S
- Pop H will require $G, I \in M$, shrinking the size of S

Overall, $S = \{B, C, G, I\}$ is the largest set of which the elements are independent of each other, conditionally on $M = \{A, D, E, F, H\}$.

(d)

Given the linear-Gaussian structure with no correlation between children, it is easily to define factor model as:

$$p(\mathcal{X}) = p(A)p(B | A)p(C | A)p(D | C)p(F | B, D)p(E | C)p(G)p(H | E, G)p(I | F, H)$$

where $p(A) \sim \mathcal{N}(0, 1)$ and $p(G) \sim \mathcal{N}(0, 1)$. The conditional distribution is in the form of $\mathcal{N}(\lambda z, \psi)$. For example $B | A = \lambda_{AB}A + \psi_B\epsilon_B$ where $\epsilon_B \sim \mathcal{N}(0, 1)$.

We can then write all the variables in matrix form, putting observed variables in the front:

$$\begin{bmatrix} B \\ D \\ E \\ G \\ A \\ C \\ F \\ H \\ I \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & \lambda_{AB} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \lambda_{CD} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \lambda_{CE} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \lambda_{AC} & 0 & 0 & 0 & 0 \\ \lambda_{BF} & \lambda_{DF} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \lambda_{EH} & \lambda_{GH} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \lambda_{FI} & \lambda_{HI} & 0 \end{bmatrix} \begin{bmatrix} B \\ D \\ E \\ G \\ A \\ C \\ F \\ H \\ I \end{bmatrix} + \begin{bmatrix} \psi_B\epsilon_B \\ \psi_D\epsilon_D \\ \psi_E\epsilon_E \\ \epsilon_G \\ \epsilon_A \\ \psi_C\epsilon_C \\ \psi_F\epsilon_F \\ \psi_H\epsilon_H \\ \psi_I\epsilon_I \end{bmatrix}$$

For variables B, D, E, G , we have:

$$\begin{bmatrix} B \\ D \\ E \\ G \end{bmatrix} = \begin{bmatrix} \lambda_{AB} & 0 \\ 0 & \lambda_{CD} \\ 0 & \lambda_{CE} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} A \\ C \end{bmatrix} + \begin{bmatrix} \psi_B\epsilon_B \\ \psi_D\epsilon_D \\ \psi_E\epsilon_E \\ \epsilon_G \end{bmatrix}$$

The covariance matrix of B, D, E, G is:

$$\begin{bmatrix} \lambda_{AB}^2 + \psi_B^2 & \lambda_{AB}\lambda_{AC}\lambda_{CD} & \lambda_{AB}\lambda_{AC}\lambda_{CE} & 0 \\ \lambda_{AB}\lambda_{AC}\lambda_{CD} & \lambda_{AC}^2\lambda_{CD}^2 + \lambda_{CD}^2\psi_C^2 + \psi_D^2 & \lambda_{AC}^2\lambda_{CD}\lambda_{CE} & 0 \\ \lambda_{AB}\lambda_{AC}\lambda_{CE} & \lambda_{AC}^2\lambda_{CD}\lambda_{CE} & \lambda_{AC}^2\lambda_{CE}^2 + \lambda_{CE}^2\psi_C^2 + \psi_E^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Since we know $\delta[B], \delta[D], \delta[E], \delta[G]$, this means we know the covariance structure of those variables. We can update the posterior distribution of A, C since they are the only 2 factors influencing these observed variables. Specifically we can recover $\lambda_{AB}, \lambda_{CD}, \lambda_{CE}, \lambda_{AC}$.

(e)

For the rest variables F, H, I ,

$$F = [\lambda_{BF} \quad \lambda_{DF}] \begin{bmatrix} B \\ D \end{bmatrix} + \psi_F \epsilon_F = (\lambda_{BF} \lambda_{AB} + \lambda_{DF} \lambda_{CD} \lambda_{AC}) A + \lambda_{BF} \psi_B \epsilon_B + \lambda_{DF} \lambda_{CD} \psi_C \epsilon_C + \lambda_{DF} \psi_D \epsilon_D + \psi_F \epsilon_F$$

Observing B and D means we know the mean and covariance structure of them. We can identify the ratio of λ_{BF} and λ_{DF} based on the covariance structure of observed B, D but not the absolute values, as they are scale invariant.

Similar we can do it for λ_{EH} and λ_{GH} .

However, for I ,

$$I \mid B, D, E, G = \Lambda_I \begin{bmatrix} F \\ H \end{bmatrix} + \psi_I \epsilon_I$$

Knowing B, D, E, G does not send any message on I . The influence of F and H on I does not propagate back to any observed variables. I remain unidentifiable.

Q2

(a)

We can rewrite $f(t) = at + b + \epsilon(t)$ as:

$$y = \mathbf{w}\mathbf{x} + \epsilon(t)$$

where

$$\mathbf{w} = \begin{bmatrix} a \\ b \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} t \\ 1 \end{bmatrix}$$

$$y \sim \mathcal{N}(\mathbf{w}\mathbf{x}, 1)$$

Prior distribution is:

$$\mathbf{w} \sim \mathcal{N}(\mu, \Sigma_0)$$

where

$$\mu_0 = \begin{bmatrix} 0 \\ 360 \end{bmatrix} \quad \Sigma_0 = \begin{bmatrix} 10^2 & 0 \\ 0 & 100^2 \end{bmatrix}$$

We know the posterior distribution is

$$\mathbf{w} \mid \mathcal{D} \sim \mathcal{N}((X^\top X + \Sigma_0^{-1})^{-1}(\Sigma_0^{-1}\mu_0 + X^\top \mathbf{y}), (X^\top X + \Sigma_0^{-1})^{-1})$$

where $X \in \mathbb{R}^{n \times 2}$ and $\mathbf{y} \in \mathbb{R}^n$.

```

1 import numpy as np
2 y = co2_data['average'].to_numpy()
3 t = co2_data['decimal'].to_numpy()
4 X = np.stack((t, np.ones_like(t))).T
5 mu0 = np.array([0, 360])
6 inv_sigma0 = np.diag([0.1**2, 0.01**2])

```

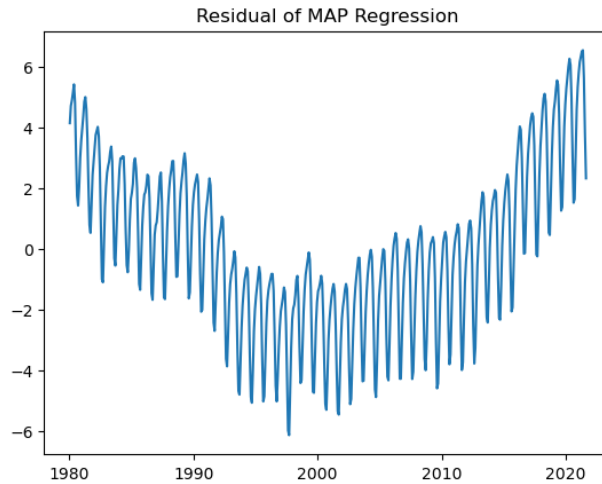


Figure 1.1. MAP Parameter Residual

```

7 sigma_pos = np.linalg.inv(X.T @ X + inv_sigma0)
8 mu_pos = sigma_pos @ (inv_sigma0 @ mu0 + X.T @ y)

```

Listing 1.1. Posterior Calculation

The outputs are as follows:

```

1 mu_pos
2 # array([ 1.81842808e+00, -3.26615096e+03])
3 sigma_pos
4 # array([[ 1.37479698e-05, -2.75073908e-02],
5 #        [-2.75073908e-02,  5.50396935e+01]])

```

Listing 1.2. Output

(b)

The plot is 1.1. The residuals do not look like the following $\mathcal{N}(0, 1)$ as we can see a clear trend of the mean values deviating from 0. If the residuals are $\mathcal{N}(0, 1)$, we should see it around 0 all the time. The variance, on the other hand, looks roughly the same as the bandwidth does not change too much over the time.

(c)

```

1 def Gaussian_Process(cov_kernel, inputs, random_samples):
2     K = cov_kernel(inputs, inputs)
3     d, V = np.linalg.eigh(K)
4     d = np.maximum(d, 1e-14)
5     L = V @ np.diag(np.sqrt(d))
6     f = L @ random_samples

```

```
return f
```

Listing 1.3. Gaussian Process Function

(d)

There could be a typo in summative problem sheet as θ is the coefficient of both periodic kernel and squared exponential kernel. We will discuss the characteristics of hyperparameters in the following kernel (separating the effect of θ on squared exponential kernel for more clear feature of the hyperparameters).

$$k(s, t) = \theta^2 \exp\left(-\frac{2 \sin^2\left(\pi \frac{s-t}{\tau}\right)}{\sigma^2}\right) + \phi^2 \exp\left(-\frac{(s-t)^2}{2\eta^2}\right) + \zeta^2 \delta_{s=t}$$

This is the implementation of the kernel.

```
def sum_exp_squared_kernel(theta, sigma, tau, phi, eta, zeta):
    return lambda s,t: theta**2 * np.exp(-2 * np.sin(np.pi * (s.
        reshape(-1,1) - t.reshape(1,-1)) / tau)**2 / sigma**2) + phi
        **2 * np.exp(-(s.reshape(-1,1) - t.reshape(1,-1))**2 / 2 /
        eta**2) + zeta**2 * np.equal(s.reshape(-1,1), t.reshape(1,-1)
        )
```

Listing 1.4. Kernel

We use same random samples with different parameters over domain $[-5, 5]$. While varying a certain hyperparameter, we keep all the rest fixed.

```
domain = np.linspace(-5, 5, 100)
thetas = [0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
sigmas = [0.01, 0.1, 0.5, 1.0, 2.0, 3.0]
taus = [0.1, 0.5, 1, 2, 5, 10]
phis = [0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
etas = [0.01, 0.1, 0.5, 1.0, 2.0, 3.0]
zetas = [0.01, 0.1, 0.5, 1.0, 2.0, 3.0]

np.random.seed(100)
random_samples = np.random.rand(domain.shape[0], 10)
theta_kernels = [sum_exp_squared_kernel(theta, 1.0, 1.0, 1.0, 1.0, 1.0) for theta in thetas]
sigma_kernels = [sum_exp_squared_kernel(1.0, sigma, 1.0, 1.0, 1.0, 1.0) for sigma in sigmas]
tau_kernels = [sum_exp_squared_kernel(1.0, 1.0, tau, 1.0, 1.0, 1.0) for tau in taus]
phi_kernels = [sum_exp_squared_kernel(1.0, 1.0, 1.0, phi, 1.0, 1.0) for phi in phis]
eta_kernels = [sum_exp_squared_kernel(1.0, 1.0, 1.0, 1.0, eta, 1.0) for eta in etas]
```

```

16 zeta_kernels = [sum_exp_squared_kernel(1.0, 1.0, 1.0, 1.0, 1.0, zeta
    ) for zeta in zetas]

```

Listing 1.5. Plot Hyperparameters

We also plot the heat map of the kernel function with different hyperparameter values. We observe the characteristics of the hyperparameters.

- θ : Increase the weight on periodic kernel over squared exponential one. GP process has stronger periodic pattern
- σ : Control the smoothness/magnitude of the periodic variations, with smaller values resulting in sharper oscillations.
- τ : Change the period of oscillation in kernel. Larger τ has less frequency
- ϕ : Weight on squared exponential kernel. Larger ϕ gives larger weight on squared exponential kernel over periodic kernel. GP shows less periodic pattern and more smooth trend
- η : Similar to σ , controlling the decay of the memory. Larger η will keep long memory.
- ζ : Coefficient of independent noise. Emphasise the kernel value on same input. The larger ζ , the larger contrast the variance over covariance

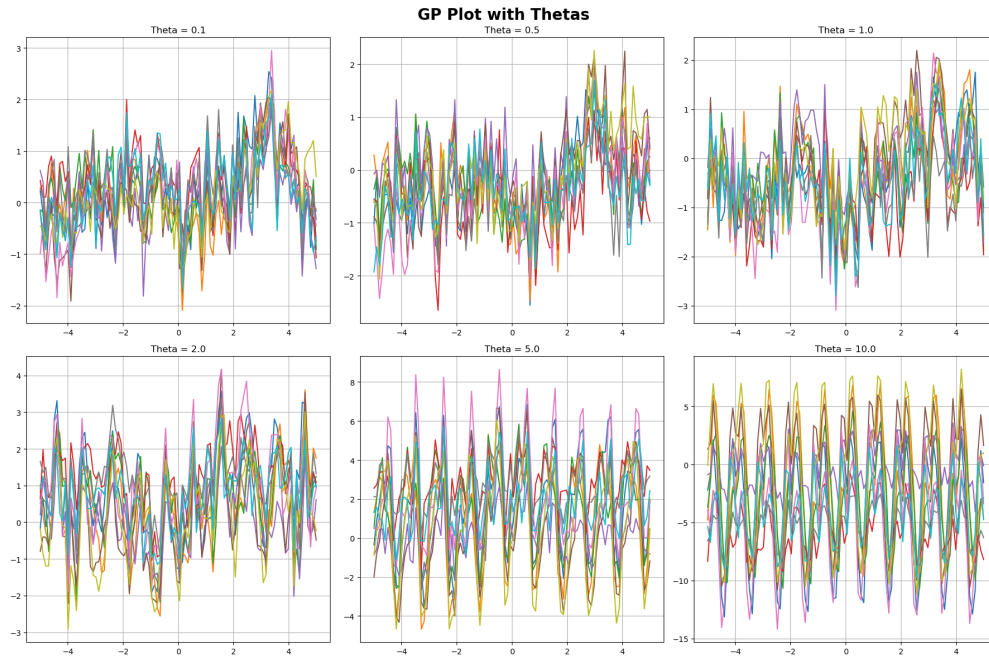
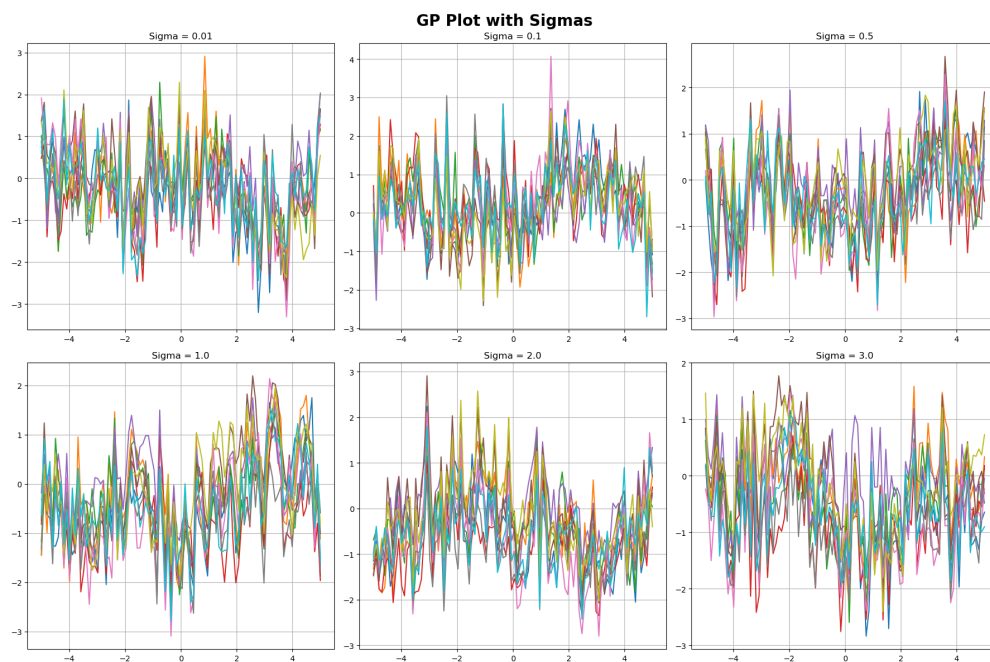
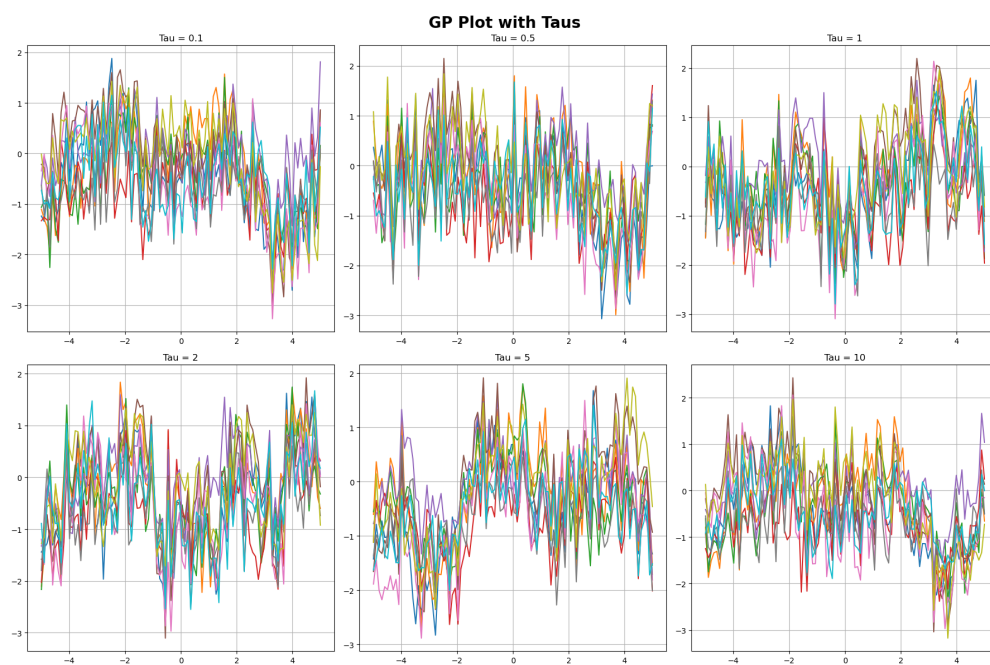
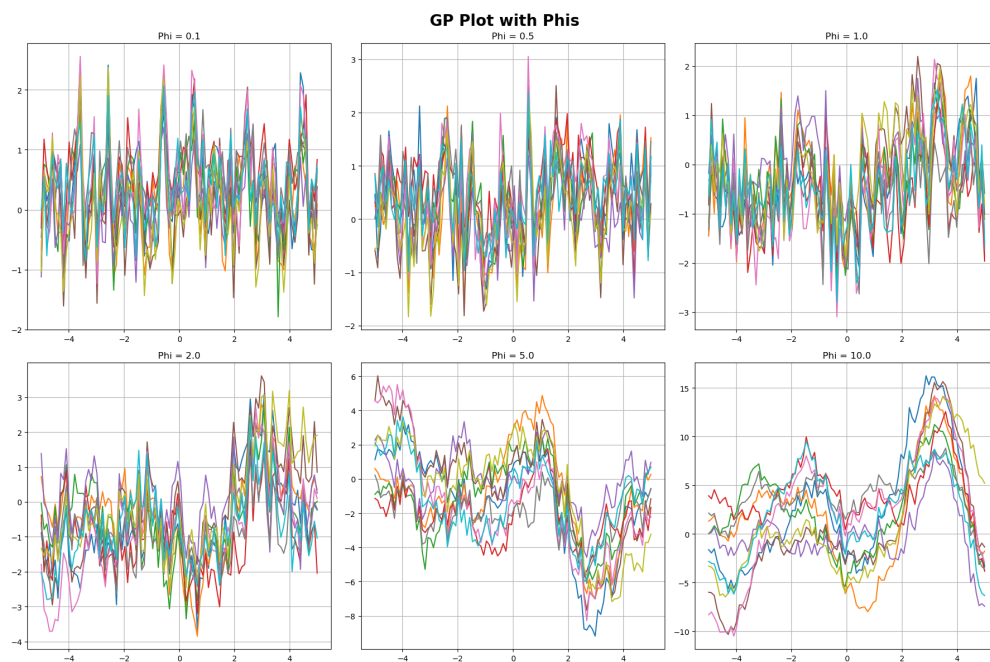
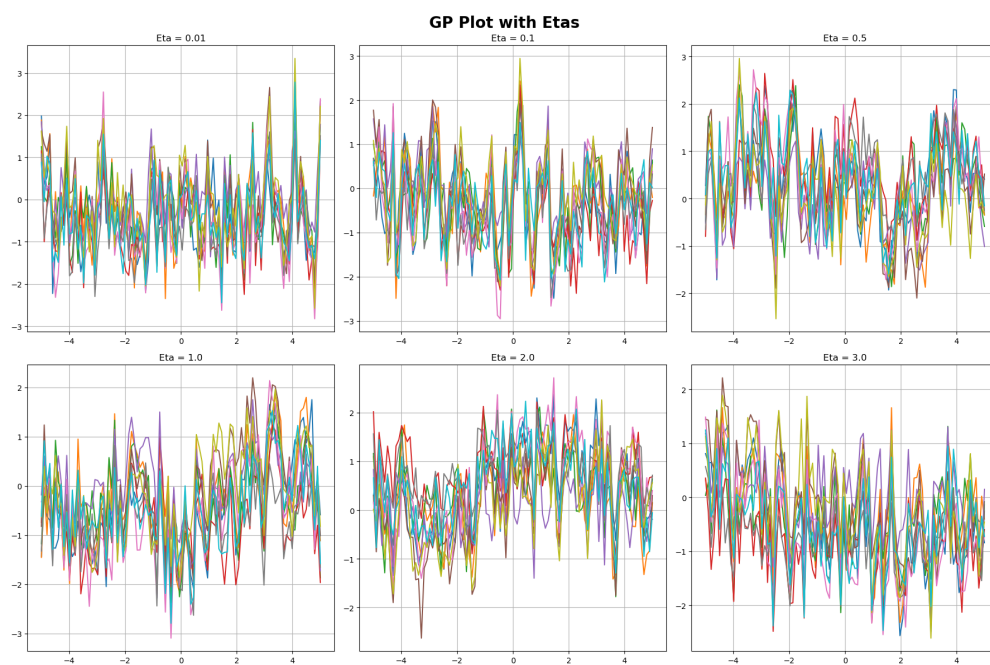
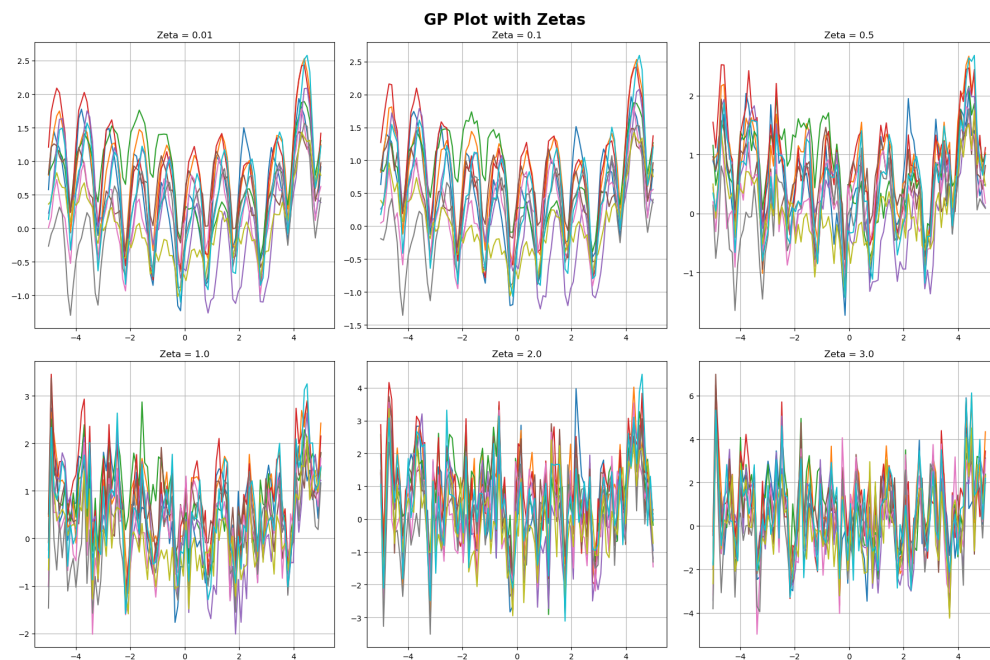
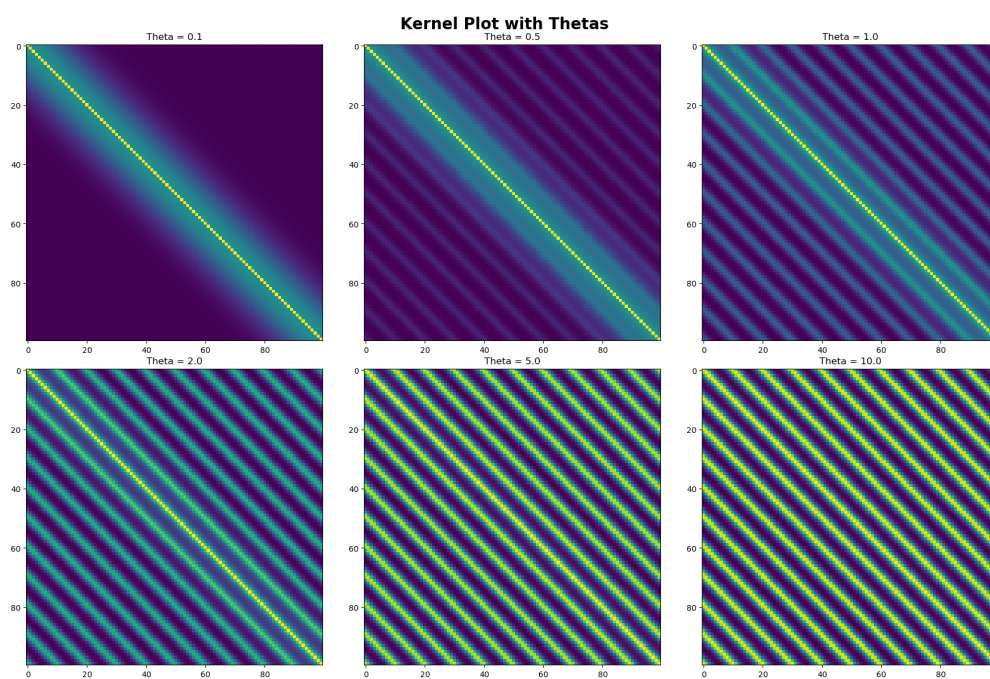
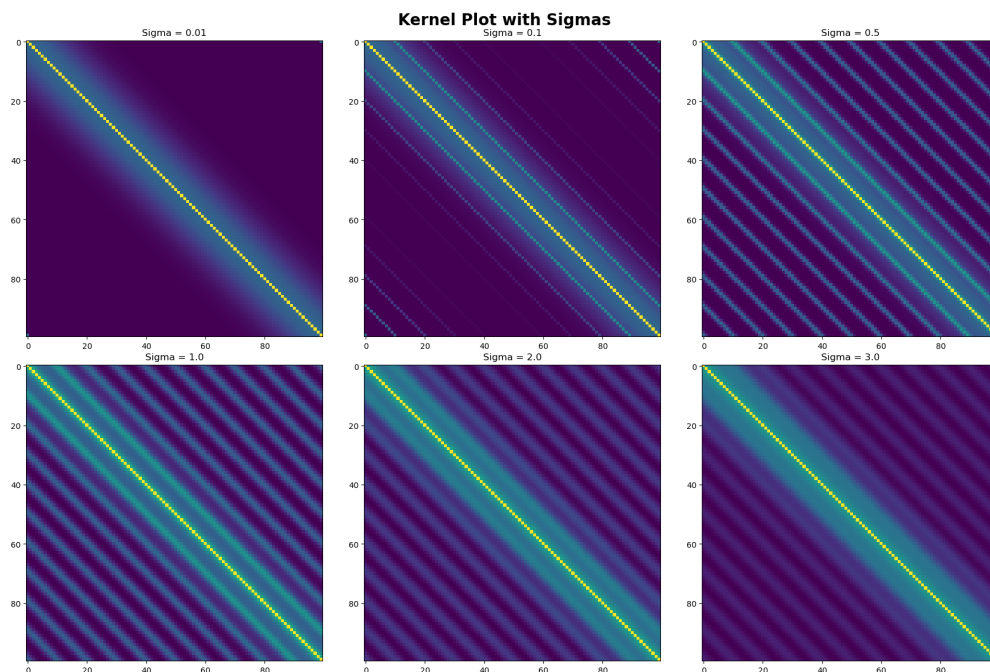
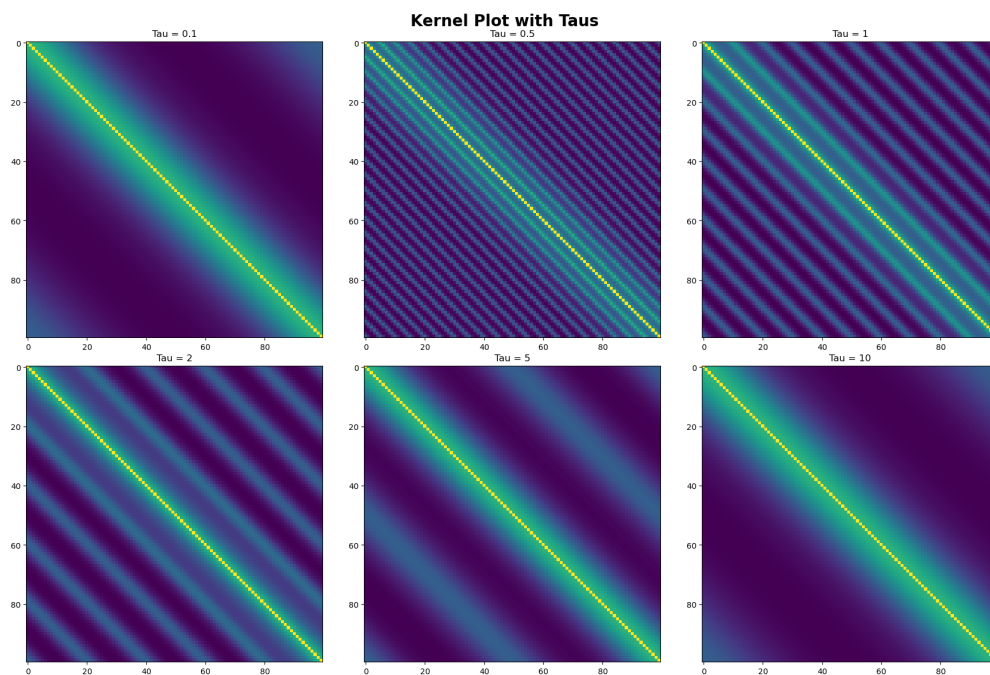


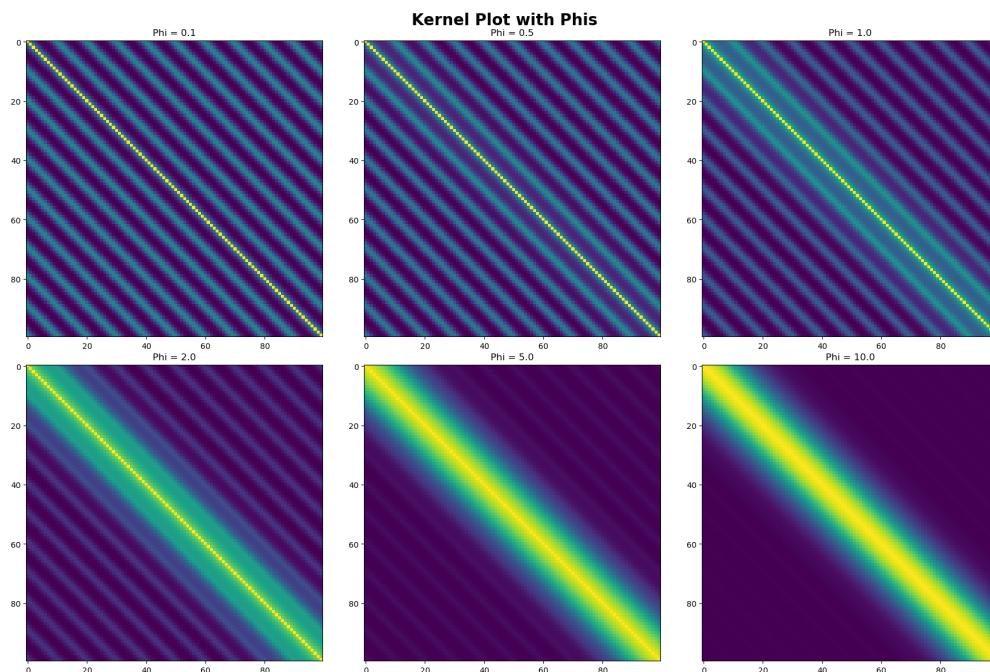
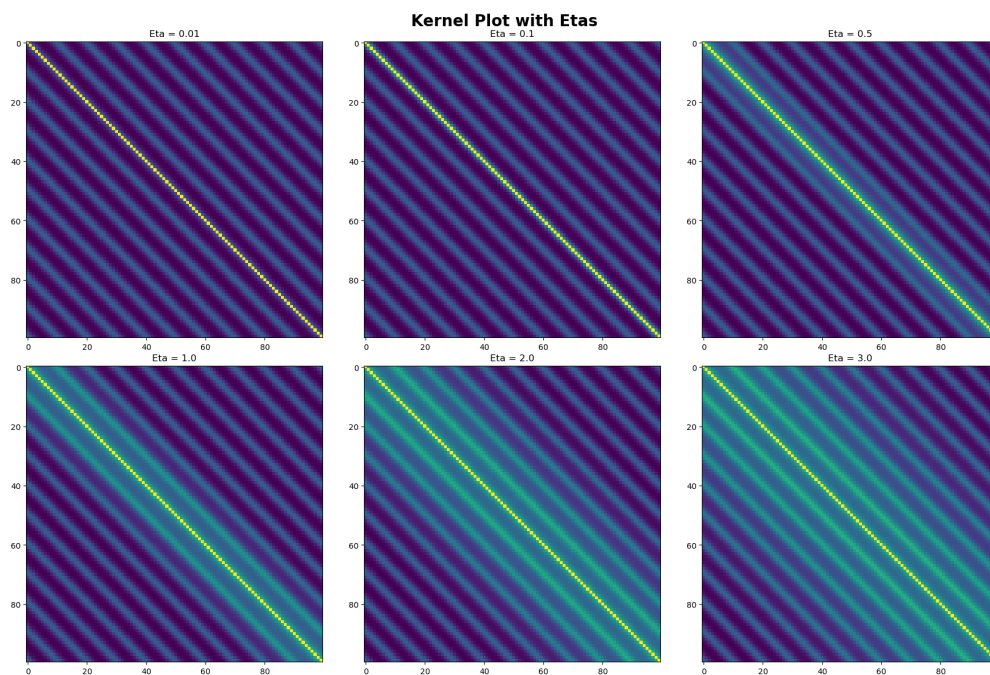
Figure 1.2. GP Plot with θ

Figure 1.3. GP Plot with σ Figure 1.4. GP Plot with τ

Figure 1.5. GP Plot with ϕ Figure 1.6. GP Plot with η

Figure 1.7. GP Plot with ζ Figure 1.8. Kernel Heat Map with θ

Figure 1.9. Kernel Heat Map with σ Figure 1.10. Kernel Heat Map with σ

Figure 1.11. Kernel Heat Map with ϕ Figure 1.12. Kernel Heat Map with η

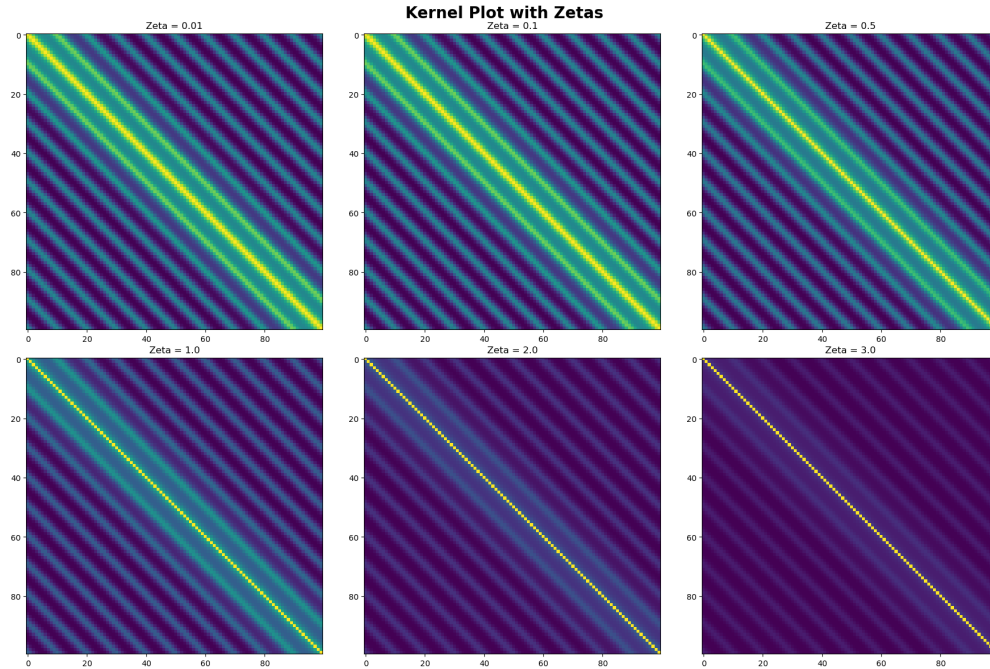


Figure 1.13. Kernel Heat Map with ζ

(e)

From the residual plot $g(t)$, we can observe the periodic pattern. There are 42 peaks in 500 data points. Roughly we estimate the periodicity as 12. So we can set $\tau = 12/12 = 1$. σ should be small so that we can pick the sharp oscillation. We will set θ significantly larger than ϕ to line up the strong periodic pattern of the data. η is set to a large value to capture smooth long-term trend. ζ is estimated from the standard deviation of the data. Set $\zeta = 2.68$. In sum, we try this set of hyperparameters:

- $\tau : 1$
- $\sigma : 0.1$
- $\eta : 2.0$
- $\theta : 10.0$
- $\phi : 1.0$
- $\zeta : 2.68$

(f)

```

1 train_data_index = co2_data[(co2_data['year'] < 2007) | ((co2_data['
   year'] == 2007) & (co2_data['month'] < 9))].index
2 train_data_x = t[train_data_index]
```

```
3 train_data_y = g_obs[train_data_index]
4 testing_data_idx = co2_data.iloc[332:]
5 testing_data_idx = testing_data_idx[testing_data_idx['year'] <
    2021].index
6 testing_data_x = t[testing_data_idx]
7 testing_data_y = g_obs[testing_data_idx]
8
9 co2_kernel = sum_exp_squared_kernel(10.0, 0.1, 1.0, 1.0, 2.0, 2.68)
10
11 KXX = co2_kernel(train_data_x, train_data_x)
12 KxX = co2_kernel(testing_data_x, train_data_x)
13 KXX_inverse = np.linalg.inv(KXX)
14 Kxx = co2_kernel(testing_data_x, testing_data_x)
15 mean = KxX @ (KXX_inverse @ train_data_y)
16 cov = Kxx - KxX @ KXX_inverse @ KxX.T
17 std = np.sqrt(np.diag(cov))
18
19 f_train = y[train_data_index]
20 f_test = y[testing_data_idx]
21
22 regression_part = X @ mu_pos
23 regression_part_testing = regression_part[testing_data_idx]
24 gp_f = regression_part_testing + mean
25
26 plt.figure()
27 plt.plot(train_data_x, f_train, label='Observed_Training_CO2')
28 plt.plot(testing_data_x, f_test, label='Observed_CO2')
29 plt.plot(testing_data_x, gp_f, label='Predicted_Mean')
30 plt.fill_between(testing_data_x, gp_f - std, gp_f + std, color='k',
    alpha=0.2)
31 plt.legend()
32 plt.show()
```

Listing 1.6. GP Prediction

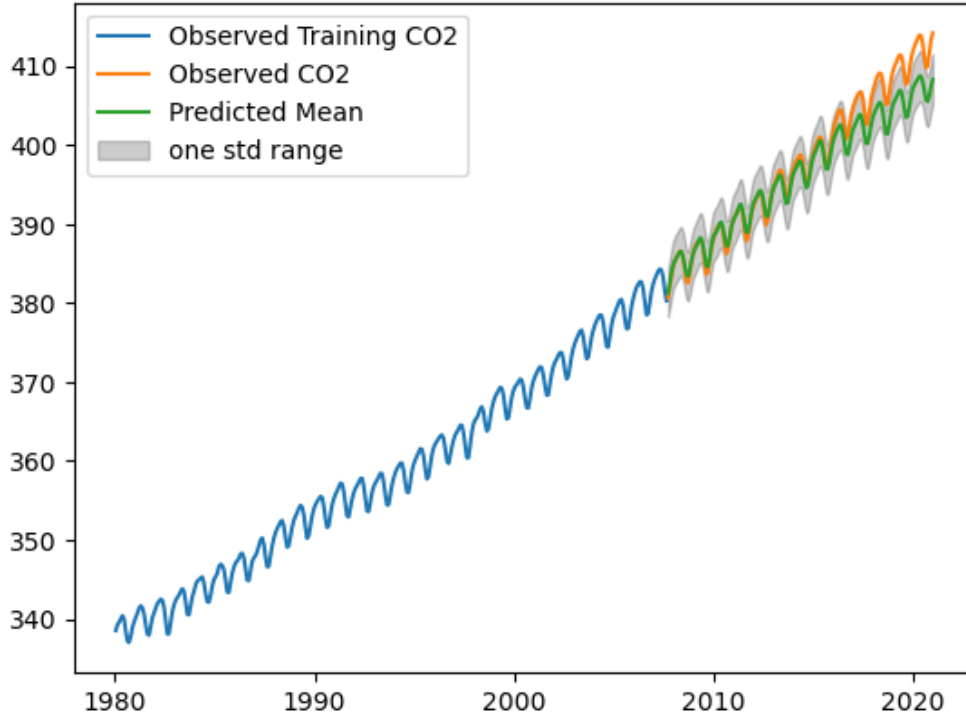


Figure 1.14. GP prediction with error estimation

Compared with the observed CO2 data, GP predicted data match very well until 2015 after when we see observed CO2 was accelerated and deviated from 1 standard deviation of the prediction. The hyperparameters are chosen suitably as we can see the periodicity of prediction matches with the observed. The amplification matches well in the beginning so our choice of σ is okay. For long term trend parameter in squared exponential kernel, we tried larger η to capture long distanced data but it does not provide better extrapolation after 2015. Besides, increasing ϕ will make the 1 standard deviation range wider.

(g)

We estimated a and b using MAP which is not Bayesian. Also we choose hyperparameters instead of getting posterior distribution of them and integrate them out in the prediction. Such method is not Bayesian as well.

To make it Bayesian, we should pose prior beliefs on parameters and do the prediction with integrating the posterior distribution on a and b .

$$p(y \mid \mathbf{x}, \mathcal{D}) = \int p(y \mid \mathbf{x}, \mathbf{w}, \mathcal{D}) p(\mathbf{w} \mid \mathcal{D}) d\mathbf{w}$$

And with hyperparameters, we can use Bayesian Occam's factor to do the hyperparameter selection. Or we can propose priors on them, integrating hyperparameters out to average the prediction.

Q3

We propose variational approximation on each data point $\mathbf{x}^{(n)}$ with mean-field approximations (fully factored):

$$q_n(\mathbf{s}^{(n)}) = \prod_{k=1}^K \lambda_{kn}^{s_k^{(n)}} \left(1 - \lambda_{kn}^{s_k^{(n)}}\right)^{(1-s_k^{(n)})} = \prod_{k=1}^K q_{kn} \left(s_k^{(n)}\right)$$

We have free energy:

$$\mathcal{F}_n \left(\prod_{k=1}^K q_{kn} \left(s_k^{(n)}\right), \theta \right) = \left\langle \log P(\mathbf{x}_n, \mathbf{s}^{(n)} \mid \theta) \right\rangle_{\prod_k q_{kn}} + \mathbf{H} \left[\prod_{k=1}^K q_{kn} \left(s_k^{(n)}\right) \right]$$

$$\begin{aligned} \log P(\mathbf{x}_n, \mathbf{s}^{(n)} \mid \theta) &= \log P(\mathbf{x}_n \mid \mathbf{s}^{(n)}, \theta) + \log P(\mathbf{s}^{(n)} \mid \theta) \\ &= -\frac{1}{2\sigma^2} \left(\mathbf{x}_n - \sum_k s_k^{(n)} \mu_k \right)^\top \left(\mathbf{x}_n - \sum_k s_k^{(n)} \mu_k \right) \\ &\quad + \sum_k s_k^{(n)} \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} + \sum_k \log \left(1 - \pi_k^{(n)} \right) + \text{const} \\ &= -\frac{1}{2\sigma^2} \left(-2 \sum_k s_k^{(n)} \mu_k^\top \mathbf{x}_n + \sum_k \left(s_k^{(n)} \right)^2 \mu_k^\top \mu_k + \sum_{i \neq j} s_i^{(n)} s_j^{(n)} \mu_i^\top \mu_j \right) \\ &\quad + \sum_k s_k^{(n)} \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} + \text{const} \end{aligned}$$

$$\begin{aligned} q_{kn} \left(s_k^{(n)} \right) &\propto \exp \left\langle \log P(\mathbf{x}_n, \mathbf{s}^{(n)} \mid \theta) \right\rangle_{\prod_{-k} q_{in}} \\ &\propto \exp \left(-\frac{1}{2\sigma^2} \left(\left(s_k^{(n)} \right)^2 \mu_k^\top \mu_k + 2 \sum_{i \neq k} \langle s_i^{(n)} \rangle_{q_{in}} s_k^{(n)} \mu_i^\top \mu_k - 2 s_k^{(n)} \mu_k^\top \mathbf{x}_n \right) + s_k^{(n)} \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} \right) \\ &= \exp \left(-\frac{1}{2\sigma^2} \left(\left(s_k^{(n)} \right)^2 \mu_k^\top \mu_k + 2 \sum_{i \neq k} \lambda_{in} s_k^{(n)} \mu_i^\top \mu_k - 2 s_k^{(n)} \mu_k^\top \mathbf{x}_n \right) + s_k^{(n)} \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} \right) \end{aligned}$$

Since $s_k^{(n)}$ is binary, we can get unnormalised $q_{kn}(1)$ and $q_{kn}(0)$ and normalise them to update λ_{in} with sigmoid function.

$$q_{kn}(1) \propto \exp \left(-\frac{1}{2\sigma^2} \left(\|\mu_k\|^2 + 2 \sum_{i \neq k} \lambda_{in} \langle \mu_i, \mu_k \rangle - 2 \langle \mu_k, \mathbf{x}_n \rangle \right) + \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} \right)$$

$$q_{kn}(0) \propto 1$$

$$\lambda_{kn} = \frac{1}{1 + \exp(-q_{kn}(1))}$$

So in E step, we update λ_{in} in turn from 1 to K .

Specifically, we have free energy as:

$$\begin{aligned} \mathcal{F}_n \left(\prod_{k=1}^K q_{kn} \left(s_k^{(n)} \right), \theta \right) = & -\frac{1}{2\sigma^2} \left(\|\mathbf{x}_n\|^2 - 2 \sum_k \lambda_{kn} \langle \mu_k^T, \mathbf{x}_n \rangle + \sum_k \lambda_{kn} \|\mu_k\|^2 + \sum_{i \neq j} \lambda_{in} \lambda_{jn} \langle \mu_i, \mu_j \rangle \right) \\ & + \sum_k \lambda_{kn} \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} + \sum_k \log \left(1 - \pi_k^{(n)} \right) \\ & - \sum_k (\lambda_{kn} \log \lambda_{kn} + (1 - \lambda_{kn}) \log(1 - \lambda_{kn})) - \frac{D}{2} \log(2\pi\sigma^2) \end{aligned}$$

(a)

The implementation of E step is:

```

1 def MeanField(X, mu, sigma, pie, lambda0, maxsteps, tol):
2     """
3     E step for mean field variational approximation for binary
4         latent factor model
5
6     Input:
7     X:          (N, D) np array, data
8     mu:          (D, K) np array, means
9     sigma:       scalar, variance of data
10    pie:          (K,) np array, prior on s
11    lambda0:      (N, K) np array, initial values of lambda
12    maxsteps:     max iteration
13    tol:          tolerance of halting the iteration
14
15    Output:
16    lmbda:        (N, K) lambda after iteration
17    F:            Free energy in each iteration
18    """
19    D, K = mu.shape
20    mu2 = mu.T @ mu # (K, K)
21    mu2_diag = np.diag(mu2)
22    mu2_offdiag = mu2 - np.diag(mu2_diag)
23    mu_x = X @ mu # (N, K)
24    log_pi = np.log(pie.flatten()) # (K, )
25    log_one_minus_pi = np.log(1 - pie.flatten()) # (K, )
26    log_pi_one_minus_pi = log_pi - log_one_minus_pi # (K, )
27    lmbda = lambda0.copy()
28    lmbda_mu_cross = np.array([lmbda_row @ mu2_offdiag @ lmbda_row.T
29                                for lmbda_row in lmbda])
30    F_n = (

```

```

29     np.sum(lmbda * log_pi_one_minus_pi, axis=1) + np.sum(
30         log_one_minus_pi) # Prior term
31     - np.sum(lmbda * np.log(lmbda) + (1 - lmbda) * np.log(1 -
32         lmbda), axis=1) # Entropy term
33     - 0.5 / sigma**2 * (np.sum(X**2, axis=1) - 2 * np.sum(lmbda
34         * mu_x, axis=1) + lmbda_mu_cross.flatten() + np.sum(lmbda
35         * mu2_diag[None,:], axis=1)) # Likelihood term
36     - D / 2 * np.log(2 * np.pi * sigma**2) # Normalisation term
37 )
38 F = np.sum(F_n)
39 Fs = [F]
40 for _ in range(maxsteps):
41     old_F = F
42     for k in range(K):
43         lmbda_mu_without_k = lmbda @ mu2[:, k] - lmbda[:, k] *
44             mu2[k, k]
45         logit_1 = -0.5 / sigma**2 * (mu2[k, k] + 2.0 *
46             lmbda_mu_without_k - 2.0 * mu_x[:, k]) +
47             log_pi_one_minus_pi[k]
48         lmbda[:, k] = np.clip(1.0 / (1.0 + np.exp(-logit_1)), 1e
49             -8, 1 - 1e-8)
50
51     lmbda_mu_cross = np.array([lmbda_row @ mu2_offdiag @
52         lmbda_row.T for lmbda_row in lmbda])
53
54     F_n = (
55         np.sum(lmbda * log_pi_one_minus_pi, axis=1) + np.sum(
56             log_one_minus_pi) # Prior term
57         - np.sum(lmbda * np.log(lmbda) + (1 - lmbda) * np.log(1
58             - lmbda), axis=1) # Entropy term
59         - 0.5 / sigma**2 * (np.sum(X**2, axis=1) - 2 * np.sum(
60             lmbda * mu_x, axis=1) + lmbda_mu_cross.flatten() + np
61             .sum(lmbda * mu2_diag[None,:], axis=1)) # Likelihood
62             term
63         - D / 2 * np.log(2 * np.pi * sigma**2) # Normalisation
64             term
65     )
66
67     F = np.sum(F_n)
68     Fs.append(F)
69     if np.abs(F - old_F) < tol:
70         break
71 return lmbda, Fs

```

Listing 1.7. Mean Field E Step

(b)

 $\{\mu_i\}_{i=1}^K$ only appears in

$$-\frac{1}{2\sigma^2} \left\langle \mathbf{x}_n - \sum_k s_k^{(n)} \mu_k \right\rangle_{\prod_k q_{kn}}$$

which resembles the normal equations of linear regression. We are doing linear regression of s against $\mathbf{x}$ and μ is the coefficient we would like to solve.

In standard linear regression we would like to fit $y = X\mathbf{w}$ by minimising $\|y - X\mathbf{w}\|^2$ and the solution would be solving the linear system of $X^\top X\mathbf{w} = X^\top y$. Replacing y with $\mathbf{x}_n$ and X with $s^{(n)}$, we resemble the M step for μ .

(c)

To calculate μ , we need:

- Inversion $\mathcal{O}(K^3)$
- Multiplication $\mathcal{O}(K^2N)$
- Multiplication $\mathcal{O}(KND)$

To calculate σ , we need:

- $\mathcal{O}(ND^2 + D + K^2D + K^3 + K + KND + K^2D + K) = \mathcal{O}(K^3 + ND^2 + K^2D + KND)$

To calculate π , we need $\mathcal{O}(NK)$

(d)

The images look like blend of black-white feature images, which means the features are some white patterns on black canvas. Looks like there are 8 features (1 white cross on top right corner, 1 white first row, 1 white second column, 1 white fourth column, 1 diagonal, 1 square on bottom left, 1 large unfilled square on the edges and 1 small filled square in the centre).

If we refer factor analysis as continuous Gaussian latent factors, it is not a good choice here as the latent factors (features) are in binary states.

In ICA we have:

$$x_d = \sum_{k=1}^K \Lambda_{dk} z_k + \varepsilon_d$$

and z_k is not Gaussian. If we let z_k be binary variable and $\varepsilon_d \sim \mathcal{N}(0, \sigma^2)$, this is essentially same as our binary latent factor model.

Mixture of Gaussian could be a choice, with each mixture corresponding to a possible linear combination of features. Suppose we have K features, potentially we should have 2^K mixtures. If K is large, we could have difficulty in learning the model as the number of mixture increases exponentially.

(e)

The EM algorithm is implemented as follows:

```

1 def LearnBinFactors(X, K, iterations, sigma0=None, tol=1.0e-8):
2     """
3     EM algorithm
4
5     Input:
6     X: (N, D) data
7     K: scalar, number of binary factors
8     iterations: scalar, max iterations
9     sigma0: scalar, initial value of sigma
10    tol: scalar, tolerance of halting the iteration
11    """
12    N, D = X.shape
13    pie_init = np.empty(K); pie_init.fill(0.5)
14    mu_init = np.random.randint(0, 2, (D, K))
15    if sigma0:
16        sigma_init = sigma0
17    else:
18        sigma_init = 0.5 * K
19    lambda_init = np.tile(pie_init, (N, 1))
20    Fs = []
21    pie = pie_init.copy()
22    mu = mu_init.copy()
23    sigma = sigma_init
24    lambda = lambda_init.copy()
25    for _ in range(iterations):
26        lambda, M_Fs = MeanField(X, mu, sigma, pie, lambda, 100, 1.0e-8)
27        Fs.append(M_Fs[-1])
28        ES = lambda
29        ESS = np.array([this_lambda.reshape(-1, 1) * this_lambda.
30                        reshape(1, -1) for this_lambda in lambda])
31        for n in range(N):
32            np.fill_diagonal(ESS[n, :, :], lambda[n])
33        mu, sigma, pie = m_step(X, ES, ESS)
34        if len(Fs) > 1 and np.abs(Fs[-1] - Fs[-2]) < tol:
35            break
36    return mu, sigma, pie, Fs

```

Listing 1.8. Mean Field EM

(f)

μ should be K black-white 4×4 images which form the basis of the data. The data should be linear combination of those basis with weights 0 or 1. We can first put large K and check

the learnt feature images, eliminate the seemingly same feature to reduce K and relearn. We choose $K = 8$ here.

For parameter initialisation, we do not have much knowledge on initial probability of hidden feature. So we set all of them 0.5. For μ we can randomly choose 0 or 1 to form an $D \times K$ matrix. λ is set to be same as π for each n . σ is the white noise of linear combination of K features. We initialise it as $0.5 \times K$ to line up the initialisation of π so that within $\pm\sigma$ we should have 0 or all features.

We can improve the algorithm by adding prior beliefs on parameters to have a Bayesian way. We can also use automatic relevance determination to select number of K (see Q4).

To make feature more distinctive, we can make it have higher contrast by exaggerating light gray (> 0.5) to classify as white and dark gray (< 0.5) to be pure black. This will present more clear feature map.

```
1 np.random.seed(1994)
2 K = 8
3 mu, sigma, pie, Fs = LearnBinFactors(Y, K, 200)
4 plt.plot(Fs)
5 plt.title("Free Energy in Each Iteration")
6
7 for k in range(K):
8     plt.subplot(4, 4, k + 1)
9     plt.imshow(np.reshape(mu.T[k], (4, 4)), cmap=plt.gray(),
10                interpolation='none')
11     plt.axis('off')
12 plt.show()
```

Listing 1.9. Run EM

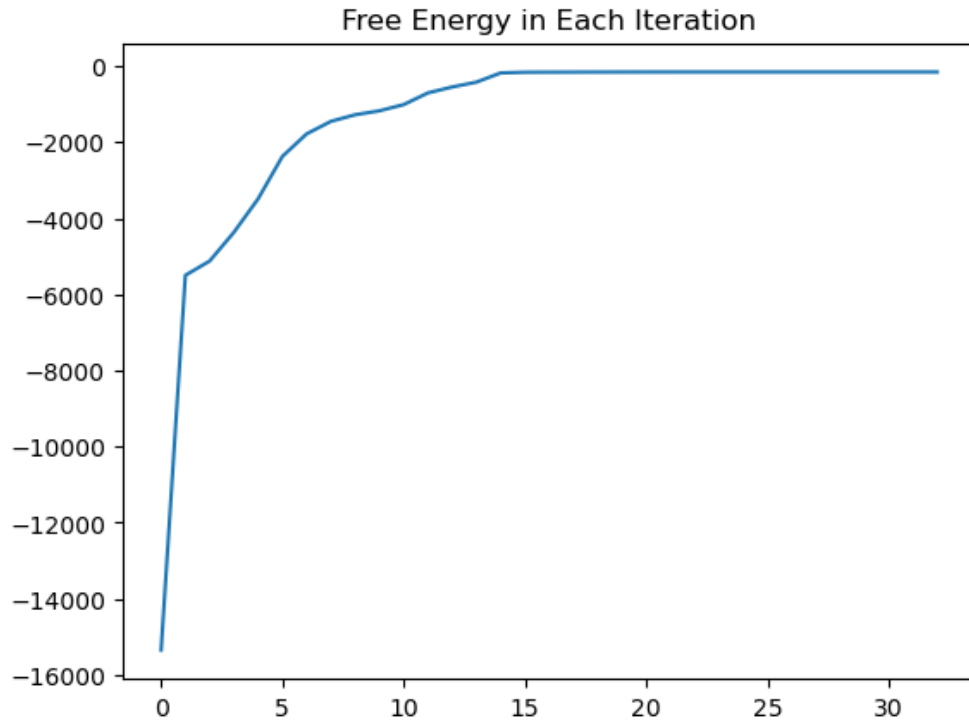


Figure 1.15. Free Energy in Iterations

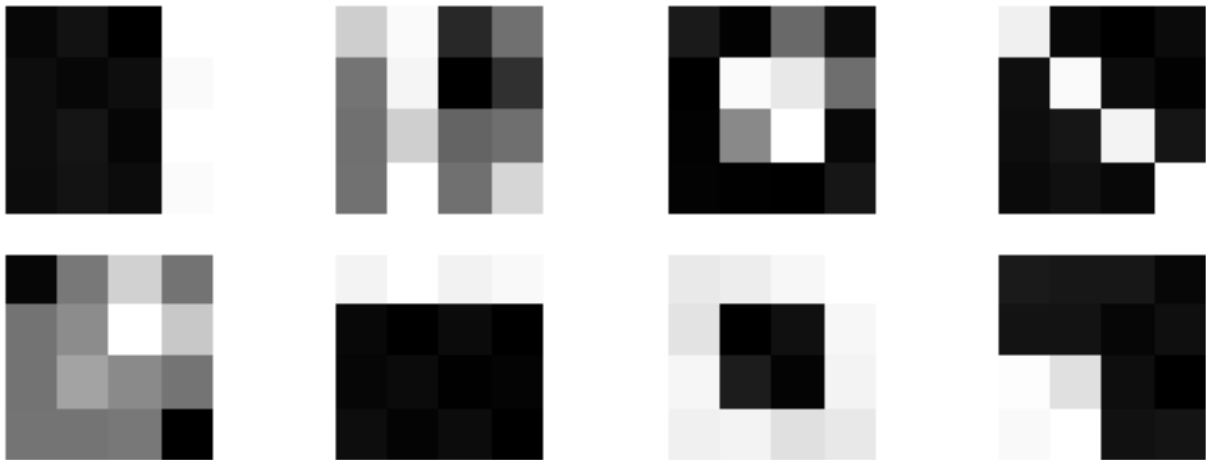


Figure 1.16. Features from EM

(g)

We set $K = 4$ because one single image probably does not have many features embedding. We choose $\sigma = 0.5, 1, 5$. The iteration converges very fast. Generally speaking, λ converges

faster for larger σ . Recall that

$$q_{kn}(1) \propto \exp \left(-\frac{1}{2\sigma^2} \left(\|\mu_k\|^2 + 2 \sum_{i \neq k} \lambda_{in} \langle \mu_i, \mu_k \rangle - 2 \langle \mu_k, \mathbf{x}_n \rangle \right) + \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} \right)$$

We have then:

$$\frac{\partial q_{kn}(1)}{\partial \lambda_{in}} \propto -\frac{q_{kn}(1) \langle \mu_i, \mu_k \rangle}{\sigma^2}$$

This means whenever we update $\lambda_{in}, \forall i \neq k$, smaller σ will make larger q_{kn} (and λ_{kn}) movement. Therefore it takes more iterations to reach a stable mean field posterior probability.

Another observation is that smaller σ produces higher free energy. σ measures the noise in the data. The smaller the noise, the larger contribution of latent features on explaining the data which results higher evidence lower bound.

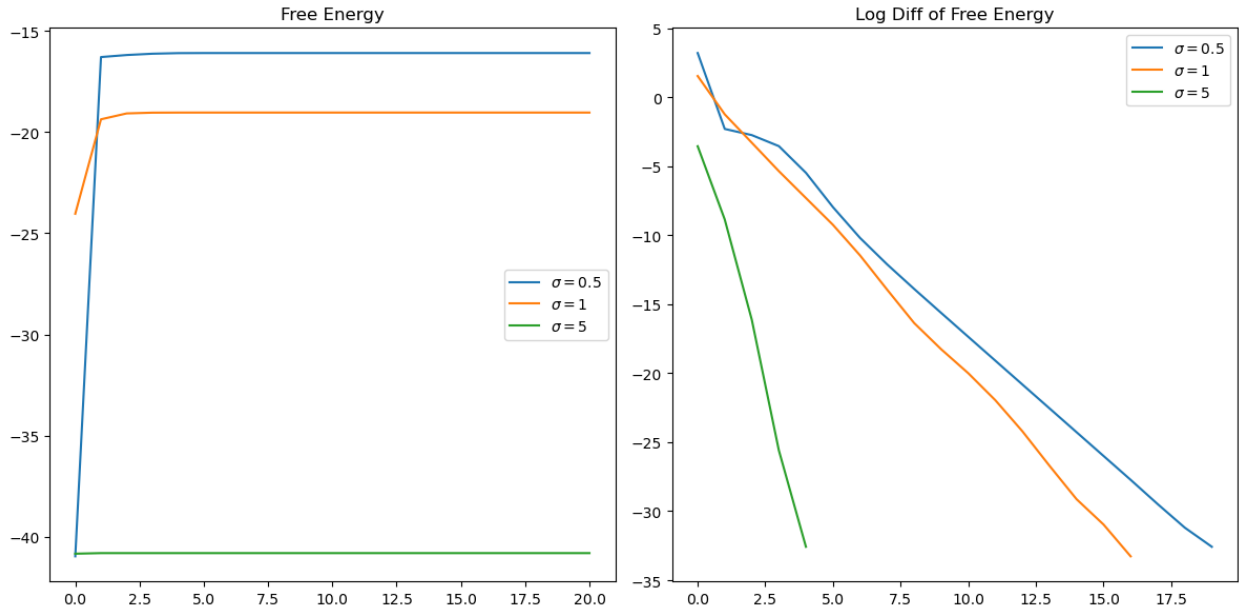


Figure 1.17. Single Data E step with different sigma

Q4

(a)

We can write

$$p(\mathbf{x} \mid s_1, \dots, s_K, \Lambda, \sigma) = \mathcal{N}(\Lambda \mathbf{s}, \sigma^2 I)$$

where

$$\Lambda := \begin{bmatrix} | & | & \dots & | \\ \mu_1 & \mu_2 & \dots & \mu_K \\ | & | & \dots & | \end{bmatrix}$$

We can impose a prior distribution of Λ , with each column following distribution

$$\Lambda_{:i} \sim \mathcal{N}(0, \alpha_i^{-1} I)$$

Also we impose Beta distribution as prior on π_i , each π_i shares same hyperparameter α_π and β_π .

$$\pi_i \sim \text{Beta}(\alpha_\pi, \beta_\pi)$$

We impose inverse Gamma distribution as prior on σ^2

$$\sigma^2 \sim \text{Inv}\Gamma(\alpha_\sigma, \beta_\sigma)$$

We have hyperparameter set:

$$\eta = \{\alpha_1, \dots, \alpha_K, \alpha_\pi, \beta_\pi, \alpha_\sigma, \beta_\sigma\}$$

The factorisation of approximate posterior is $\prod_n q_n(s^{(n)}) q_\Lambda(\Lambda | \eta) q_\pi(\pi | \eta) q_{\sigma^2}(\sigma^2 | \eta)$. The VB free energy is then:

$$\begin{aligned} \mathcal{F} & \left(\prod_n q_n(\mathbf{s}^{(n)}), q_\Lambda(\Lambda), q_\pi(\pi), q_{\sigma^2}(\sigma^2), \eta \right) \\ &= \left\langle \sum_n \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} | \Lambda, \sigma^2, \pi, \eta) + \log P(\Lambda | \eta) + \log P(\pi | \eta) + \log P(\sigma^2 | \eta) \right\rangle_{\prod q_n(\mathbf{s}^{(n)}) q_\Lambda(\Lambda) q_\pi(\pi) q_{\sigma^2}(\sigma^2)} \\ &+ \mathbf{H} \left[\prod_n q_n(\mathbf{s}^{(n)}) q_\Lambda(\Lambda) q_\pi(\pi) q_{\sigma^2}(\sigma^2) \right] \end{aligned}$$

with

$$q_{\Lambda_{:i}}(\Lambda_{:i}) \sim \mathcal{N}(\Lambda_{:i} | m_i, V_i)$$

$$q_\pi(\pi) \sim \prod_{i=1}^K \text{Beta}(\alpha_{\pi,i}^m, \beta_{\pi,i}^m)$$

$$q_{\sigma^2}(\sigma^2) \sim \text{Inv}\Gamma(\alpha_{\sigma^2}^m, \beta_{\sigma^2}^m)$$

Expectations of sufficient statistics are:

- $\langle \Lambda_{:i} \rangle = m_i \quad \langle \Lambda_{:i}^\top \Lambda_{:i} \rangle = m_i^\top m_i + \text{Tr}(V_i)$
- $\langle \log \pi_i \rangle = \psi(\alpha_{\pi,i}^m) - \psi(\alpha_{\pi,i}^m + \beta_{\pi,i}^m) \quad \langle \log(1 - \pi_i) \rangle = \psi(\beta_{\pi,i}^m) - \psi(\alpha_{\pi,i}^m + \beta_{\pi,i}^m)$
- $\langle \log(\sigma^2) \rangle = \log(\beta_\sigma^m) - \psi(\alpha_\sigma^m) \quad \langle (\sigma^2)^{-1} \rangle = \frac{\alpha_\sigma^m}{\beta_\sigma^m}$

where

$$\psi(x) := \frac{d}{dx} \log \Gamma(x)$$

Free energy is calculated as:

$$\begin{aligned}
\mathcal{F} & \left(\prod_n^N q_n(\mathbf{s}^{(n)}), q_\Lambda(\Lambda), q_\pi(\pi), q_{\sigma^2}(\sigma^2), \eta \right) \\
&= \left\langle \sum_n^N \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} \mid \Lambda, \sigma^2, \pi, \eta) \right\rangle_{\prod q_n(\mathbf{s}^{(n)}) q_\Lambda(\Lambda) q_\pi(\pi) q_{\sigma^2}(\sigma^2)} + \langle \log P(\Lambda \mid \eta) \rangle_{q_\Lambda(\Lambda)} + \langle \log P(\pi \mid \eta) \rangle_{q_\pi(\pi)} \\
&+ \langle \log P(\sigma^2 \mid \eta) \rangle_{q_{\sigma^2}(\sigma^2)} + \mathbf{H} \left[\prod_n^N q_n(\mathbf{s}^{(n)}) q_\Lambda(\Lambda) q_\pi(\pi) q_{\sigma^2}(\sigma^2) \right] \\
&= -\frac{ND}{2} (\log(\beta_\sigma^m) - \psi(\alpha_\sigma^m)) - \frac{\alpha_\sigma^m}{2\beta_\sigma^m} \sum_n^N \left\langle (\mathbf{x}^{(n)} - \Lambda \mathbf{s}^{(n)})^\top (\mathbf{x}^{(n)} - \Lambda \mathbf{s}^{(n)}) \right\rangle_{\prod q_n q_\Lambda} \\
&+ \sum_n^N \sum_i^K (\lambda_{ni} (\psi(\alpha_{\pi,i}^m) - \psi(\alpha_{\pi,i}^m + \beta_{\pi,i}^m)) + (1 - \lambda_{ni}) (\psi(\beta_{\pi,i}^m) - \psi(\alpha_{\pi,i}^m + \beta_{\pi,i}^m))) \\
&+ \frac{1}{2} \sum_i^K (D \log(\alpha_i) - \alpha_i (m_i^\top m_i + \text{Tr}(V_i))) \\
&+ (\alpha_\pi - 1) \sum_i^K (\psi(\alpha_{\pi,i}^m) - \psi(\alpha_{\pi,i}^m + \beta_{\pi,i}^m)) + (\beta_\pi - 1) \sum_i^K (\psi(\beta_{\pi,i}^m) - \psi(\alpha_{\pi,i}^m + \beta_{\pi,i}^m)) \\
&- (\alpha_\sigma + 1) (\log(\beta_\sigma^m) - \psi(\alpha_\sigma^m)) - \frac{\alpha_\sigma^m \beta_\sigma}{\beta_\sigma^m} \\
&- \sum_n^N \sum_i^K (\lambda_{ni} \log \lambda_{ni} + (1 - \lambda_{ni}) \log(1 - \lambda_{ni})) \\
&+ \frac{1}{2} \sum_i^K \log \det(V_i) \\
&+ \sum_i^K (\log B(\alpha_{\pi,i}^m, \beta_{\pi,i}^m) - (\alpha_{\pi,i}^m - 1) \psi(\alpha_{\pi,i}^m) - (\beta_{\pi,i}^m - 1) \psi(\beta_{\pi,i}^m) + (\alpha_{\pi,i}^m + \beta_{\pi,i}^m - 2) \psi(\alpha_{\pi,i}^m + \beta_{\pi,i}^m)) \\
&+ \alpha_\sigma^m + \log \beta_\sigma^m + \log \Gamma(\alpha_\sigma^m) - (1 + \alpha_\sigma^m) \psi(\alpha_\sigma^m) + \text{const}
\end{aligned}$$

The red part is updated in E-step. So we have:

- E-step: $q_n^*(\mathbf{s}^{(n)}) \propto \exp \left\langle \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} \mid \Lambda, \sigma^2, \pi, \eta) \right\rangle_{q_\Lambda^* q_\pi^* q_{\sigma^2}^*}$
- M-step:
 - $q_\Lambda^*(\Lambda) \propto p(\Lambda \mid \eta) \exp \left\langle \sum_n \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} \mid \Lambda, \sigma^2, \pi, \eta) \right\rangle_{\prod q_n^*(\mathbf{s}^{(n)}) q_\pi^*(\pi) q_{\sigma^2}^*(\sigma^2)}$
 - $q_\pi^*(\pi) \propto p(\pi \mid \eta) \exp \left\langle \sum_n \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} \mid \Lambda, \sigma^2, \pi, \eta) \right\rangle_{\prod q_n^*(\mathbf{s}^{(n)}) q_\Lambda^*(\Lambda) q_{\sigma^2}^*(\sigma^2)}$

$$- q_{\sigma^2}^*(\sigma^2) \propto p(\sigma^2 \mid \eta) \exp \left\langle \sum_n \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} \mid \Lambda, \sigma^2, \pi, \eta) \right\rangle_{\prod q_n^*(\mathbf{s}^{(n)}) q_\Lambda^*(\Lambda) q_\pi^*(\pi)}$$

- hyperparameter optimisation requires:

$$- \alpha_i \leftarrow \arg \max \langle \log P(\Lambda \mid \alpha_i) \rangle_{q_\Lambda}$$

Note we have:

$$\begin{aligned} & \log P(\mathbf{x}, \mathbf{s} \mid \Lambda, \sigma^2, \pi, \eta) \\ &= -\frac{D}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{x} - \Lambda \mathbf{s})^\top (\mathbf{x} - \Lambda \mathbf{s}) + \sum_i s_i \log \frac{\pi_i}{1 - \pi_i} + \sum_i \log(1 - \pi_i) + \text{const} \end{aligned}$$

E-step:

$$\begin{aligned} & \left\langle \log P(\mathbf{x}, \mathbf{s} \mid \Lambda, \sigma^2, \pi, \eta) \right\rangle_{q_\Lambda^*(\Lambda) q_\pi^*(\pi) q_{\sigma^2}^*(\sigma^2)} \\ &= -\frac{D}{2} \left\langle \log(\sigma^2) \right\rangle_{q_{\sigma^2}^*} - \frac{1}{2} \left\langle \frac{1}{\sigma^2} \right\rangle_{q_{\sigma^2}^*} \left\langle (\mathbf{x} - \Lambda \mathbf{s})^\top (\mathbf{x} - \Lambda \mathbf{s}) \right\rangle_{q_\Lambda^*} + \sum_i s_i \left\langle \log \frac{\pi_i}{1 - \pi_i} \right\rangle_{q_\pi^*} + \text{const} \\ &= -\frac{D}{2} (\log(\beta_\sigma^m) - \psi(\alpha_\sigma^m)) - \frac{\alpha_\sigma^m}{2\beta_\sigma^m} (-2\mathbf{x}^\top \langle \Lambda \rangle_{q_\Lambda} \mathbf{s} + \mathbf{s}^\top \langle \Lambda^\top \Lambda \rangle_{q_\Lambda} \mathbf{s}) + \sum_i s_i (\psi(\alpha_{\pi,i}^m) - \psi(\beta_{\pi,i}^m)) + \text{const} \end{aligned}$$

So we have:

$$\log q_{ni}^*(s_i^{(n)}) = s_i^{(n)} \left(\frac{\alpha_\sigma^m \mathbf{x}^{(n)\top} m_i}{\beta_\sigma^m} + \psi(\alpha_{\pi,i}^m) - \psi(\beta_{\pi,i}^m) - \frac{\alpha_\sigma^m (m_i^\top m_i + \text{Tr}(V_i))}{2\beta_\sigma^m} - \frac{\alpha_\sigma^m \sum_{j \neq i} \lambda_{nj} m_j^\top m_i}{\beta_\sigma^m} + \text{const} \right)$$

$$\lambda_{ni} = q(s_i^{(n)} = 1) \propto \exp \left(\frac{\alpha_\sigma^m \mathbf{x}^{(n)\top} m_i}{\beta_\sigma^m} + \psi(\alpha_{\pi,i}^m) - \psi(\beta_{\pi,i}^m) - \frac{\alpha_\sigma^m (m_i^\top m_i + \text{Tr}(V_i))}{2\beta_\sigma^m} - \frac{\alpha_\sigma^m \sum_{j \neq i} \lambda_{nj} m_j^\top m_i}{\beta_\sigma^m} \right)$$

$$1 - \lambda_{ni} = q(s_i^{(n)} = 0) \propto 1$$

M-step:

For $q_{\Lambda_{:i}}$, we have:

$$\begin{aligned} & \left\langle \sum_n \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} \mid \Lambda, \sigma, \pi, \eta) \right\rangle_{\prod q_n \prod_{-i} q_{\Lambda_{:j}} q_\pi q_{\sigma^2}} + \log P(\Lambda_{:i} \mid \eta) \\ &= -\frac{\alpha_\sigma^m}{2\beta_\sigma^m} \sum_n \left\langle (\mathbf{x}^{(n)} - \Lambda \mathbf{s}^{(n)})^\top (\mathbf{x}^{(n)} - \Lambda \mathbf{s}^{(n)}) \right\rangle_{\prod q_n(\mathbf{s}^{(n)}) \prod_{-i} q_{\Lambda_{:j}}} - \frac{1}{2} \alpha_i \Lambda_{:i}^\top \Lambda_{:i} + \text{const} \\ &= -\frac{\alpha_\sigma^m}{2\beta_\sigma^m} \sum_n \left\langle \mathbf{x}^{(n)\top} \mathbf{x}^{(n)} - 2\mathbf{x}^{(n)\top} \Lambda_{:i} s_i^{(n)} + \Lambda_{:i}^\top \Lambda_{:i} s_i^{(n)} + 2 \sum_{j \neq i} \Lambda_{:j}^\top \Lambda_{:i} s_j^{(n)} s_i^{(n)} \right\rangle_{\prod q_n(\mathbf{s}) \prod_{-i} q_\Lambda} - \frac{1}{2} \alpha_i \Lambda_{:i}^\top \Lambda_{:i} + \text{const} \\ &= -\frac{\alpha_\sigma^m}{2\beta_\sigma^m} \left(\sum_n \lambda_{ni} \Lambda_{:i}^\top \Lambda_{:i} - 2 \sum_n \left(\mathbf{x}^{(n)\top} \lambda_{ni} \Lambda_{:i} - \sum_{j \neq i} \lambda_{nj} \lambda_{ni} m_j^\top \Lambda_{:i} \right) \right) - \frac{1}{2} \alpha_i \Lambda_{:i}^\top \Lambda_{:i} + \text{const} \end{aligned}$$

So we have

$$q_{\Lambda_{:,i}}^*(\Lambda_{:,i}) \sim \mathcal{N}(\Lambda_{:,i} \mid m_i, V_i)$$

where

$$V_i = \left(\frac{\alpha_\sigma^m \sum_n \lambda_{ni}}{\beta_\sigma^m} + \alpha_i \right)^{-1} I_D$$

$$m_i = V_i \frac{\alpha_\sigma^m \sum_n \left(\lambda_{ni} \mathbf{x}^{(n)} - \sum_{j \neq i} \lambda_{ni} \lambda_{nj} m_j \right)}{\beta_\sigma^m}$$

To update α , we take derivative:

$$\nabla \langle \log P(\Lambda \mid \alpha) \rangle_{q_\Lambda} = \begin{bmatrix} \frac{D}{2\alpha_1} - \frac{1}{2} \langle \Lambda_{:,1}^\top \Lambda_{:,1} \rangle_{q_\Lambda} \\ \vdots \\ \frac{D}{2\alpha_K} - \frac{1}{2} \langle \Lambda_{:,K}^\top \Lambda_{:,K} \rangle_{q_\Lambda} \end{bmatrix} = \begin{bmatrix} \frac{D}{2\alpha_1} - \frac{1}{2} (m_1^\top m_1 + \text{Tr}(V_1)) \\ \vdots \\ \frac{D}{2\alpha_K} - \frac{1}{2} (m_K^\top m_K + \text{Tr}(V_K)) \end{bmatrix}$$

and solve it to give:

$$\alpha_i^* = \frac{D}{m_i^\top m_i + \text{Tr}(V_i)}$$

For q_π , we have:

$$\begin{aligned} & \left\langle \sum_n \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} \mid \Lambda, \sigma, \pi, \eta) \right\rangle_{\prod q_{s(n)} q_\Lambda q_\sigma} + \log P(\pi \mid \eta) \\ &= \sum_n \sum_i \lambda_{ni} \log \frac{\pi_i}{1 - \pi_i} + N \sum_i \log(1 - \pi_i) + \sum_i (\alpha_\pi - 1) \log(\pi_i) + \sum_i (\beta_\pi - 1) \log(1 - \pi_i) + \text{const} \\ &= \sum_i \left(\alpha_\pi - 1 + \sum_n \lambda_{ni} \right) \log(\pi_i) + \sum_i \left(\beta_\pi - 1 + N - \sum_n \lambda_{ni} \right) \log(1 - \pi_i) + \text{const} \end{aligned}$$

So we have

$$q_\pi^*(\pi) \sim \prod_{i=1}^K \text{Beta} \left(\alpha_\pi + \sum_n \lambda_{ni}, \beta_\pi + N - \sum_n \lambda_{ni} \right)$$

$$\alpha_{\pi,i}^m = \alpha_\pi + \sum_n \lambda_{ni}$$

$$\beta_{\pi,i}^m = \beta_\pi + N - \sum_n \lambda_{ni}$$

For q_{σ^2} , we have:

$$\begin{aligned}
& \left\langle \sum_n \log P(\mathbf{x}^{(n)}, \mathbf{s}^{(n)} \mid \Lambda, \sigma, \pi, \eta) \right\rangle_{\prod q_n q_\Lambda q_\pi} + \log P(\sigma^2 \mid \eta) \\
&= -\frac{ND}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} \sum_n \left\langle (\mathbf{x}^{(n)} - \Lambda \mathbf{s}^{(n)})^\top (\mathbf{x}^{(n)} - \Lambda \mathbf{s}^{(n)}) \right\rangle_{\prod q_n q_\Lambda} - (\alpha_\sigma + 1) \log(\sigma^2) - \beta_\sigma \frac{1}{\sigma^2} + \text{const} \\
&= -\left(\alpha_\sigma + \frac{ND}{2} + 1\right) \log(\sigma^2) - \left(\beta_\sigma + \frac{1}{2} \sum_n \left\langle (\mathbf{x}^{(n)} - \Lambda \mathbf{s}^{(n)})^\top (\mathbf{x}^{(n)} - \Lambda \mathbf{s}^{(n)}) \right\rangle_{\prod q_n q_\Lambda}\right) \frac{1}{\sigma^2} + \text{const} \\
&= -\left(\alpha_\sigma + \frac{ND}{2} + 1\right) \log(\sigma^2) - \\
&\quad \left(\beta_\sigma + \frac{1}{2} \sum_n \left(\mathbf{x}^{(n)\top} \mathbf{x}^{(n)} - 2\mathbf{x}^{(n)\top} \sum_i \lambda_{ni} m_i + \sum_i \lambda_{ni} (m_i^\top m_i + \text{Tr}(V_i)) + 2 \sum_{i < j} \lambda_{ni} \lambda_{nj} m_i^\top m_j \right) \right) \frac{1}{\sigma^2} + \text{const}
\end{aligned}$$

So we have

$$q_{\sigma^2}^*(\sigma^2) \sim \text{Inv}\Gamma\left(\alpha_\sigma + \frac{ND}{2}, \beta_\sigma + \frac{1}{2} \sum_n \left(\mathbf{x}^{(n)\top} \mathbf{x}^{(n)} - 2\mathbf{x}^{(n)\top} \sum_i \lambda_{ni} m_i + \sum_i \lambda_{ni} (m_i^\top m_i + \text{Tr}(V_i)) + 2 \sum_{i < j} \lambda_{ni} \lambda_{nj} m_i^\top m_j \right) \right)$$

$$\boxed{\alpha_\sigma^m = \alpha_\sigma + \frac{ND}{2}}$$

$$\boxed{\beta_\sigma^m = \beta_\sigma + \frac{1}{2} \sum_n \left(\mathbf{x}^{(n)\top} \mathbf{x}^{(n)} - 2\mathbf{x}^{(n)\top} \sum_i \lambda_{ni} m_i + \sum_i \lambda_{ni} (m_i^\top m_i + \text{Tr}(V_i)) + 2 \sum_{i < j} \lambda_{ni} \lambda_{nj} m_i^\top m_j \right)}$$

(b)

The code implementation of VB is as follows:

```

1 class VB_Mean_Field:
2     """
3     Variational Bayesian Hyperparameter optimisation algorithm to
4     automatically
5     determine K, the number of hidden binary variables
6     """
7     def __init__(self, data, K, alpha, pi_alpha, pi_beta,
8         sigma_alpha, sigma_beta):
9         """
10         data: (N, D) np array, data
11         K : Number of hidden binary variables
12         alpha: (K,) prior precision of column Lambda

```

```
11     pi_alpha, pi_beta: prior of pi ~ Beta(pi_alpha, pi_beta)
12     sigma_alpha, sigma_beta: prior of sigma^2 ~ InvGamma(
13         sigma_alpha, sigma_beta)
14     """
15     self.X = data.copy()
16     self.N, self.D = data.shape
17     self.K = K
18     self.alpha = alpha.copy()
19     self.pi_alpha = pi_alpha
20     self.pi_beta = pi_beta
21     self.sigma_alpha = sigma_alpha
22     self.sigma_beta = sigma_beta
23     self.reset()
24
25 def reset(self):
26     np.random.seed(123)
27     # alpha_opt is the optimised alpha based on ARD
28     self.alpha_opt = self.alpha.copy()
29
30     # lambda is the q_s in each iteration, (N, K)
31     self.lmbda = np.random.rand(self.N, self.K)
32
33     # mu_lambda: (D, K) np array, mean of q_Lambda
34     self.mu_lambda = np.random.rand(self.D, self.K)
35
36     # tr_var_lambda: (K, ) np array, trace of V_i of
37     # q_Lambda
38     self.tr_var_lambda = self.D / self.alpha
39
40     # q_pi_alpha: (K, ) np array, alpha for q_pi
41     self.q_pi_alpha = np.empty(self.K)
42     self.q_pi_alpha.fill(self.pi_alpha)
43
44     # q_pi_beta: (K, ) np array, beta for q_pi
45     self.q_pi_beta = np.empty(self.K)
46     self.q_pi_beta.fill(self.pi_beta)
47
48     # q_sigma_alpha: scalar, alpha for q_sigma
49     self.q_sigma_alpha = self.sigma_alpha
50
51     # q_sigma_beta: scalar, beta for q_sigma
52     self.q_sigma_beta = self.sigma_beta
53
54     self.Free_energy = 0.0
55
56 def E_step(self, maxsteps, tol):
```

```

56     """
57     E step for VB
58
59     Output:
60     Fs:          Free energy in each iteration
61     """
62
63     m2 = self.mu_lambda.T @ self.mu_lambda # (K, K)
64     m2_diag = np.diag(m2) # (K, )
65     m2_offdiag = m2 - np.diag(m2_diag)
66     m_x = self.X @ self.mu_lambda # (N, K)
67     digamma_alpha_beta = scipy.special.digamma(self.q_pi_alpha)
68     - scipy.special.digamma(self.q_pi_beta) # (K, )
69     partial_logit = self.q_sigma_alpha / self.q_sigma_beta * m_x
70     + digamma_alpha_beta[None, :] - self.q_sigma_alpha /
71     self.q_sigma_beta / 2.0 * (m2_diag[None, :] + self.
72     tr_var_lambda[None, :]) # (N, K)
73
74     lambda_mu_cross = np.array([lambda_row @ m2_offdiag @
75     lambda_row.T for lambda_row in self.lambda])
76
77     F_n = (
78         np.sum(self.lambda * digamma_alpha_beta, axis=1) # Prior
79         term
80         - np.sum(self.lambda * np.log(self.lambda) + (1 - self.
81         lambda) * np.log(1 - self.lambda), axis=1) # Entropy
82         term
83         - 0.5 * self.q_sigma_alpha / self.q_sigma_beta * (-2 *
84         np.sum(self.lambda * m_x, axis=1) + lambda_mu_cross.
85         flatten() + np.sum(self.lambda * (m2_diag[None, :] +
86         self.tr_var_lambda[None, :]), axis=1)) # Likelihood
87         term without ||x||^2
88     )
89     F = np.sum(F_n)
90     Fs = [F]
91
92     for _ in range(maxsteps):
93         old_F = F
94         for k in range(self.K):
95             lambda_mu_without_k = self.lambda @ m2_offdiag[:, k] #
96             (N, )
97             logit_1 = partial_logit[:, k] - self.q_sigma_alpha /
98             self.q_sigma_beta * lambda_mu_without_k
99             self.lambda[:, k] = np.clip(1.0 / (1.0 + np.exp(-
100             logit_1)), 1e-8, 1 - 1e-8)
101
102             lambda_mu_cross = np.array([lambda_row @ m2_offdiag @

```

```

lambda_row.T for lambda_row in self.lambda])
88
89     F_n = (
90         np.sum(self.lambda * digamma_alpha_beta, axis=1) #
            Prior term
91         - np.sum(self.lambda * np.log(self.lambda) + (1 - self
            .lambda) * np.log(1 - self.lambda), axis=1) #
            Entropy term
92         - 0.5 * self.q_sigma_alpha / self.q_sigma_beta * (-2
            * np.sum(self.lambda * m_x, axis=1) +
            lambda_mu_cross.flatten() + np.sum(self.lambda * (
            m2_diag[None, :] + self.tr_var_lambda[None, :]),
            axis=1)) # Likelihood term without ||x||^2
93     )
94
95     F = np.sum(F_n)
96     Fs.append(F)
97     if np.abs(F - old_F) < tol:
98         break
99     return Fs
100
101
102 def M_step(self):
103     """
104     M for VB
105     """
106     # Update \Lambda
107     precisions = self.q_sigma_alpha / self.q_sigma_beta * np.sum
        (self.lambda, axis=0) + self.alpha_opt
108     self.tr_var_lambda = 1.0 / precisions
109     sum_x_lambda = []
110     for this_X, this_lambda in zip(self.X, self.lambda):
111         tmp_x_lambda = this_X[:, None] * this_lambda[None, :]
112         sum_x_lambda.append(tmp_x_lambda)
113     sum_x_lambda = np.sum(np.array(sum_x_lambda), axis=0)
114
115     # iterate multiple times to update mu_lambda
116     for _ in range(10):
117         for i in range(self.K):
118             cross_terms = self.lambda @ self.mu_lambda.T - self.
                lambda[:, i].reshape(-1,1) @ self.mu_lambda[:, i].
                reshape(1,-1) # (N, D)
119             cross_terms = np.sum(cross_terms * self.lambda[:, i
                ][:, None], axis=0) # (D, )
120             self.mu_lambda[:, i] = self.q_sigma_alpha / self.
                q_sigma_beta * (sum_x_lambda[:, i] - cross_terms)
                * (1.0 / precisions[i])

```

```

121
122     # Optimise \alpha
123     m2 = self.mu_lambda.T @ self.mu_lambda # (K, K)
124     m2_diag = np.diag(m2) # (K, )
125     m2_offdiag = m2 - np.diag(m2_diag)
126     m_x = self.X @ self.mu_lambda # (N, K)
127     lambda_mu_cross = np.array([lambda_row @ m2_offdiag @
128                                lambda_row.T for lambda_row in self.lambda])
129
130     self.alpha_opt = self.D / (m2_diag + self.tr_var_lambda +
131                               1.0e-6)
132
133     # Update \pi
134     self.q_pi_alpha = self.pi_alpha + np.sum(self.lambda, axis=0)
135     self.q_pi_beta = self.pi_beta + self.N - np.sum(self.lambda,
136                                                       axis=0)
137
138     # Update \sigma^2
139     x2 = self.X @ self.X.T # (N, N)
140     x2_diag = np.diag(x2)
141     self.q_sigma_alpha = self.sigma_alpha + (self.N * self.D) /
142     2.0
143     self.q_sigma_beta = self.sigma_beta + 0.5 * np.sum(x2_diag
144     -2 * np.sum(self.lambda * m_x, axis=1) + lambda_mu_cross.
145     flatten() + np.sum(self.lambda * (m2_diag[None, :] + self.
146     tr_var_lambda[None, :]), axis=1))
147
148     digamma_alpha = scipy.special.digamma(self.q_pi_alpha)
149     digamma_beta = scipy.special.digamma(self.q_pi_beta)
150     digamma_both = scipy.special.digamma(self.q_pi_alpha + self.
151     q_pi_beta)
152     digamma_sigma_alpha = scipy.special.digamma(self.
153     q_sigma_alpha)
154     self.Free_energy = (
155         -self.N * self.D / 2 * (np.log(self.q_sigma_beta) -
156         digamma_sigma_alpha)
157         + 0.5 * np.sum(self.D * np.log(self.alpha_opt) - self.
158         alpha_opt * (m2_diag + self.tr_var_lambda))
159         + (self.pi_alpha - 1) * np.sum(digamma_alpha -
160         digamma_both) + (self.pi_beta - 1) * np.sum(
161         digamma_beta - digamma_both)
162         - (self.sigma_alpha + 1) * (np.log(self.q_sigma_beta) -
163         digamma_sigma_alpha) - self.q_sigma_alpha * self.
164         sigma_beta / self.q_sigma_beta
165         + 0.5 * np.sum(self.D * np.log(self.tr_var_lambda))
166         + np.sum(scipy.special.betaln(self.q_pi_alpha, self.
167         q_pi_beta) - (self.q_pi_alpha - 1) * digamma_alpha -
168         (self.q_pi_beta - 1) * digamma_beta + (self.

```

```

151         q_pi_alpha + self.q_pi_beta - 2) * digamma_both)
152     + self.q_sigma_alpha + np.log(self.q_sigma_beta) + scipy
153       .special.gammaln(self.q_sigma_alpha) - (1 + self.
154         q_sigma_alpha) * digamma_sigma_alpha
155     + np.sum(self.lmbda * (digamma_alpha - digamma_both))
156     - np.sum(self.lmbda * np.log(self.lmbda) + (1 - self.
157       lmbda) * np.log(1 - self.lmbda))
158     - 0.5 * self.q_sigma_alpha / self.q_sigma_beta * np.sum(
159       x2_diag -2 * np.sum(self.lmbda * m_x, axis=1) +
160       lmbda_mu_cross.flatten() + np.sum(self.lmbda * (
161         m2_diag[None, :] + self.tr_var_lambda[None, :]), axis
162         =1))
163     )
164
165 def run(self, max_iter, max_E_iter, E_tol, tol):
166     self.reset()
167     Fs_E = []
168     Fs_all = []
169     for _ in range(max_iter):
170         this_Fs = self.E_step(max_E_iter, E_tol)
171         Fs_E.append(this_Fs[-1])
172         self.M_step()
173         Fs_all.append(self.Free_energy)
174         if len(Fs_all) > 1 and np.abs(Fs_all[-1] / Fs_all[-2] -
175           1) < tol:
176             break
177     return Fs_E, Fs_all

```

Listing 1.10. VB for Binary Factors

We use the following hyperparameters:

$$\alpha_i = 2.0$$

$$\alpha_\pi = 1.0$$

$$\beta_\pi = 1.0$$

$$\alpha_\sigma = 1.01$$

$$\beta_\sigma = 1.0$$

The choice of α_π, β_π are on the basis that we have weak prior knowledge on π . The choice of α_σ reflects that we have weak prior knowledge on σ^2 but we would like to have heavy tails. β_σ is small to reflect that we believe the noise σ^2 is not large.

We choose $K = 4, 6, 8, 10, 12, 14, 16$ to run VB with the following code. We choose the effective α_i when $\alpha_i < 10$. When K is smaller than the true number of factors, the effective factors are the same as input factors. When $K > 8$, there is some pruning of effective factors. There can be still some duplicated factors which VB fails to prune. Generally speaking, free energy is larger whenever there are more effective factors. For effective factors larger than

8, the algorithm might not be able to eliminate one from the other, or blend them with each contributing some free energy. For reference, we plot the factors when $K = 16$ in 1.20, the ineffective factors have $\alpha_i > 10^7$ and the effective factors look similar to 1.16 in Q3.

```

1 pi_alpha0 = 1.0
2 pi_beta0 = 1.0
3 sigma_alpha0 = 1.01
4 sigma_beta0 = 1
5
6 Ks = np.arange(4, 17, 2)
7 F_collection = []
8 effective_alphas = []
9 for this_K in Ks:
10     alpha0 = np.ones(this_K) * 2
11     EM_VB = VB_Mean_Field(Y, this_K, alpha0, pi_alpha0, pi_beta0,
12                             sigma_alpha0, sigma_beta0)
13     F_E, F_all = EM_VB.run(500, 100, 1.0e-8, 1.0e-8)
14     F_collection.append(F_all)
15     this_alpha_opt = EM_VB.alpha_opt
16     effective_alphas.append(np.sum(np.log10(this_alpha_opt) < 1))

```

Listing 1.11. Run with different K

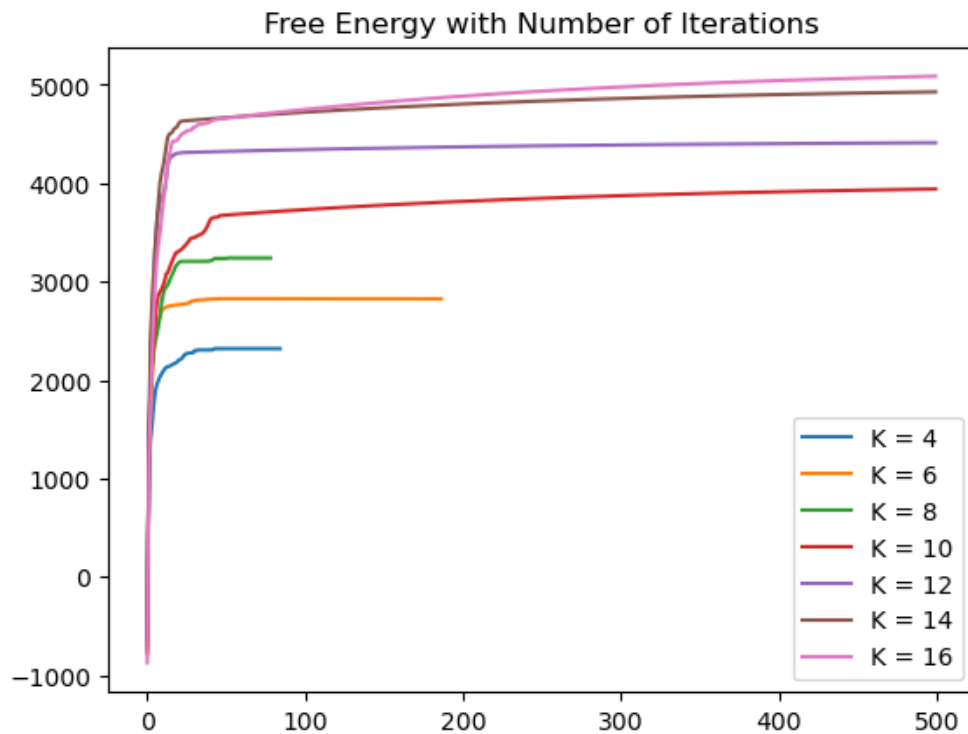


Figure 1.18. VB Free Energy with iterations for different K

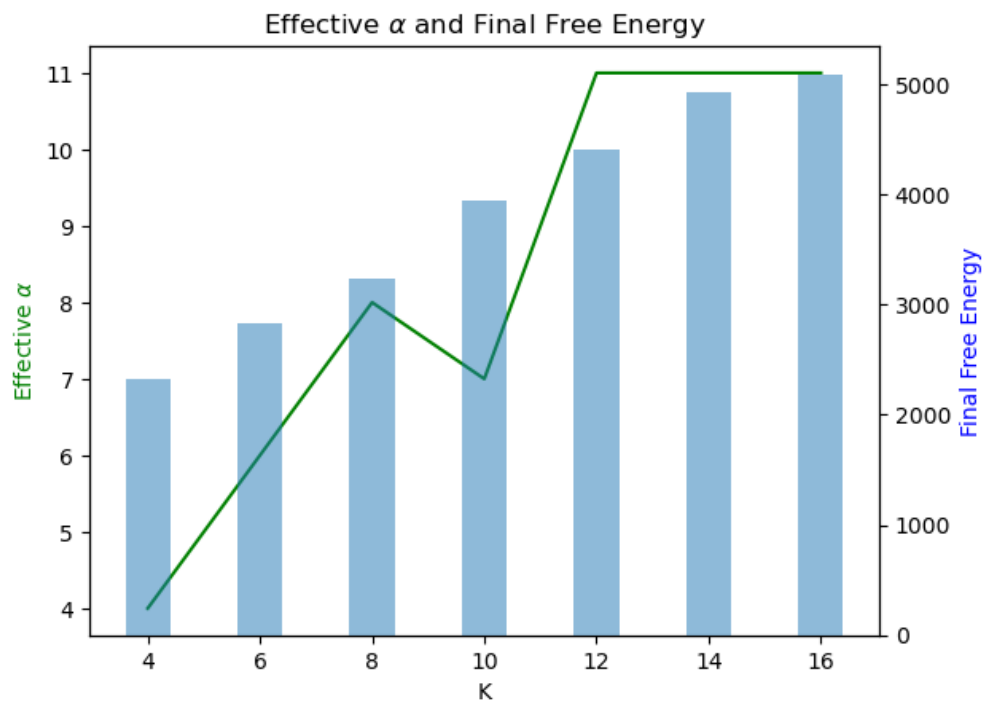


Figure 1.19. Effective α with Final Free Energy

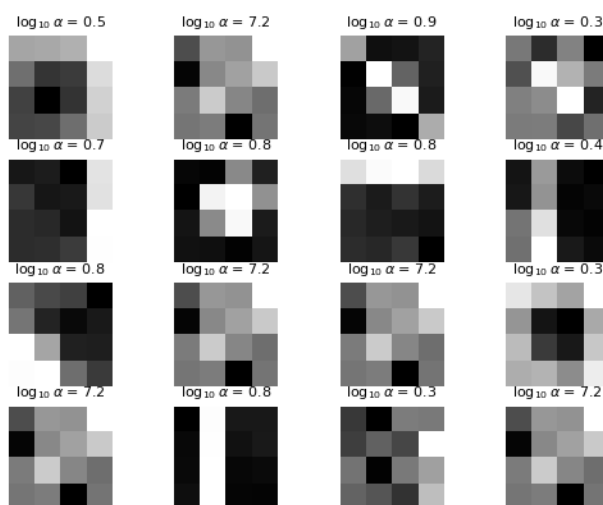


Figure 1.20. Effective Factors when $K = 16$

Q5

(a)

We have derived this in Q3. Recall:

$$\begin{aligned}
\log P(\mathbf{x}_n, \mathbf{s}^{(n)} \mid \theta) &= -\frac{1}{2\sigma^2} \left(-2 \sum_k s_k^{(n)} \mu_k^\top \mathbf{x}_n + \sum_k \left(s_k^{(n)} \right)^2 \mu_k^\top \mu_k + \sum_{i \neq j} s_i^{(n)} s_j^{(n)} \mu_i^\top \mu_j \right) \\
&\quad + \sum_k s_k^{(n)} \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} + \text{const} \\
&= \sum_k \left(\frac{\mu_k^\top \mathbf{x}_n}{\sigma^2} - \frac{\|\mu_k\|^2}{2\sigma^2} + \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} \right) s_k^{(n)} - \sum_{i < j} \frac{\mu_i^\top \mu_j}{\sigma^2} s_i^{(n)} s_j^{(n)} + \text{const} \\
&= \sum_k \log \exp \left(\left(\frac{\mu_k^\top \mathbf{x}_n}{\sigma^2} - \frac{\|\mu_k\|^2}{2\sigma^2} + \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} \right) s_k^{(n)} \right) + \sum_{i < j} \log \exp \left(-\frac{\mu_i^\top \mu_j}{\sigma^2} s_i^{(n)} s_j^{(n)} \right) \\
&\quad + \text{const}
\end{aligned}$$

So we have

$$\begin{aligned}
f_k \left(s_k^{(n)} \right) &:= \exp \left(\left(\frac{\mu_k^\top \mathbf{x}_n}{\sigma^2} - \frac{\|\mu_k\|^2}{2\sigma^2} + \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} \right) s_k^{(n)} \right) \\
g_{ij} \left(s_i^{(n)}, s_j^{(n)} \right) &:= \exp \left(-\frac{\mu_i^\top \mu_j}{\sigma^2} s_i^{(n)} s_j^{(n)} \right)
\end{aligned}$$

Recall in Boltzmann Machines we have:

$$\log P(\mathbf{s}^V \mathbf{s}^H \mid W, \mathbf{b}) = \sum_{i < j} W_{ij} \mathbf{s}_i \mathbf{s}_j - \sum_i \mathbf{b}_i \mathbf{s}_i + \text{const}$$

Let

$$\mathbf{b}_k := - \left(\frac{\mu_k^\top \mathbf{x}_n}{\sigma^2} - \frac{\|\mu_k\|^2}{2\sigma^2} + \log \frac{\pi_k^{(n)}}{1 - \pi_k^{(n)}} \right)$$

and

$$W_{ij} := -\frac{\mu_i^\top \mu_j}{\sigma^2}$$

We can recover Boltzmann Machine where f_k denote the factor of node $s_k^{(n)}$ and g_{ij} as the edges between $s_i^{(n)}$ and $s_j^{(n)}$.

(b)

We drop superscript (n) for simplicity as we evaluate single observation.

$$P(\mathbf{s} \mid \mathbf{x}) \propto \prod_i f_i(s_i) \prod_{i < j} g_{ij}(s_i, s_j)$$

Consider q with same factorisation, but potentially approximated sites:

$$q(\mathbf{s}) \propto \prod_i \tilde{f}_i(s_i) \prod_{i < j} \tilde{g}_{ij}(s_i, s_j)$$

It is natural to model

$$\tilde{f}_i(s_i) := \exp(a_i s_i),$$

and

$$\tilde{g}_{ij} := \exp(b_{ij} s_i s_j).$$

In EP, we would like to minimise KL divergence:

$$\tilde{f}_i^{\text{new}}(s_i) \leftarrow \arg \min_f \mathbf{KL} [f_i(s_i) q_{-i}(s_i) \| f(s_i) q_{-i}(s_i)]$$

where

$$q_{-i}(s_i) = \prod_{j \in \text{ne}(i)} M_{j \rightarrow i}(s_i) = \prod_{j \in \text{ne}(i)} \sum_{s_j} \tilde{g}_{ij}(s_i, s_j) \tilde{f}(s_j) \prod_{k \in \text{ne}(j) \setminus i} M_{k \rightarrow j}(s_j).$$

As we can see, in the loopy graph, marginalisation is very difficult as we need to include all the factors in the message passing and they contain loops which prevent us getting all the message ready from neighbourhood before passing. We can make some independence assumption locally to break the loop.

So instead of doing

$$\tilde{g}_{ij} := \exp(b_{ij} s_i s_j),$$

we approximate the pairwise factor as

$$\tilde{g}_{ij} := \exp(b_{j \rightarrow i} s_i) \exp(b_{i \rightarrow j} s_j)$$

Therefore we have simplified message passing scheme and cavity distribution:

$$q_{-i}(s_i) = \prod_{j \in \text{ne}(i)} M_{j \rightarrow i}(s_i) = \prod_{j \in \text{ne}(i)} \sum_{s_j} \tilde{g}_{ij}(s_i, s_j) \tilde{f}(s_j).$$

$$\tilde{f}_i^{\text{new}}(s_i) \leftarrow \arg \min_f \mathbf{KL} \left[f_i(s_i) \prod_{j \in \text{ne}(i)} \sum_{s_j} \tilde{g}_{ij}(s_i, s_j) \tilde{f}(s_j) \| f(s_i) \prod_{j \in \text{ne}(i)} \sum_{s_j} \tilde{g}_{ij}(s_i, s_j) \tilde{f}(s_j) \right]$$

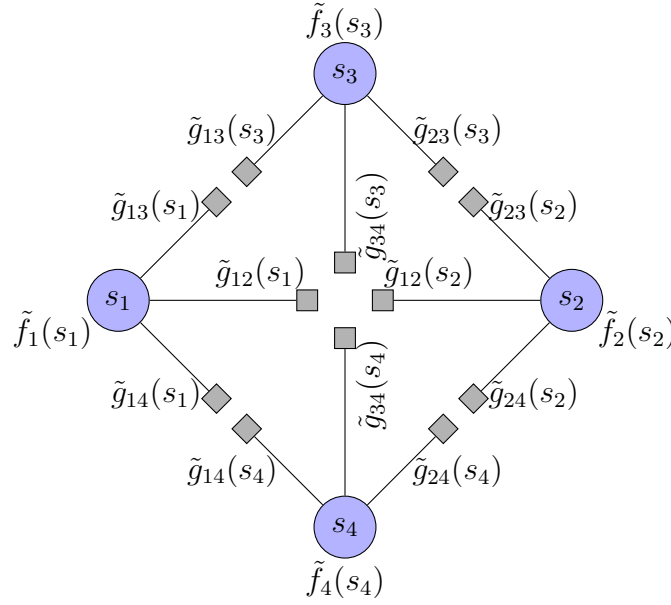
and similarly

$$\tilde{g}_{ij}^{\text{new}}(s_i, s_j) \leftarrow \arg \min_g \mathbf{KL} [g_{ij}(s_i, s_j) q_{-i,j}(s_i, s_j) \| g(s_i, s_j) q_{-i,j}(s_i, s_j)]$$

where

$$q_{-i,j}(s_i, s_j) = \tilde{f}_i(s_i) \left(\prod_{k \in \text{ne}(i) \setminus j} \sum_{s_k} \tilde{g}_{ki}(s_k, s_i) \right) \tilde{f}_j(s_j) \left(\prod_{l \in \text{ne}(j) \setminus i} \sum_{s_l} \tilde{g}_{lj}(s_l, s_j) \right)$$

From the below example with 4 factors, updating $\tilde{f}_i(s_i)$ is essentially message passing from all the neighbour pairwise factors of $s_i s_j$ and marginalising all the shared variables s_j . Updating $\tilde{g}_{ij}(s_i, s_j)$ is the product of message passing from all but $\tilde{g}_{ij}(s_i, s_j)$ to neighbour nodes $\tilde{f}_i(s_i)$, $\tilde{f}_j(s_j)$ and $\tilde{f}_i(s_i)$, $\tilde{f}_j(s_j)$ themselves.



To update a_i , we have tilted marginal distribution:

$$\begin{aligned}
 \hat{p}(s_i) &\propto f_i(s_i) \prod_{j \in \text{ne}(i)} \sum_{s_j} \tilde{g}_{ij}(s_i, s_j) \tilde{f}(s_j) \\
 &= \exp \left(\left(\frac{\mu_i^\top \mathbf{x}}{\sigma^2} - \frac{\|\mu_i\|^2}{2\sigma^2} + \log \frac{\pi_i}{1 - \pi_i} \right) s_i \right) \prod_{j \in \text{ne}(i)} \sum_{s_j=0,1} \exp(b_{j \rightarrow i} s_i) \exp(b_{i \rightarrow j} s_j) \exp(a_j s_j) \\
 &\propto \exp \left(\left(\frac{\mu_i^\top \mathbf{x}}{\sigma^2} - \frac{\|\mu_i\|^2}{2\sigma^2} + \log \frac{\pi_i}{1 - \pi_i} + \sum_{j=1, j \neq i}^K b_{j \rightarrow i} \right) s_i \right)
 \end{aligned}$$

and approximated site:

$$\begin{aligned}
 \tilde{p}(s_i) &\propto \tilde{f}_i(s_i) \prod_{j \in \text{ne}(i)} \sum_{s_j} \tilde{g}_{ij}(s_i, s_j) \tilde{f}(s_j) \\
 &\propto \exp \left(\left(a_i + \sum_{j=1, j \neq i}^K b_{j \rightarrow i} \right) s_i \right)
 \end{aligned}$$

We do moment matching of sufficient statistics (in this case s_i):

$$\langle s_i \rangle_{\tilde{p}(s_i)} = \langle s_i \rangle_{\hat{p}(s_i)}$$

Since s_i is binary, this is essentially:

$$\tilde{p}(s_i = 1) = \hat{p}(s_i = 1).$$

Therefore we have:

$$a_i = \frac{\mu_i^\top \mathbf{x}}{\sigma^2} - \frac{\|\mu_i\|^2}{2\sigma^2} + \log \frac{\pi_i}{1 - \pi_i}$$

To update b_{ij} , we have joint tilted distribution:

$$\begin{aligned}\hat{p}(s_i, s_j) &\propto g_{ij}(s_i, s_j) \tilde{f}_i(s_i) \left(\prod_{k \in ne(i) \setminus j} \sum_{s_k} \tilde{g}_{ki}(s_k, s_i) \right) \tilde{f}_j(s_j) \left(\prod_{l \in ne(j) \setminus i} \sum_{s_l} \tilde{g}_{lj}(s_l, s_j) \right) \\ &\propto \exp \left(-\frac{\mu_i^\top \mu_j}{\sigma^2} s_i s_j \right) \exp \left(\left(a_i + \sum_{k \neq i, j} b_{k \rightarrow i} \right) s_i \right) \exp \left(\left(a_j + \sum_{l \neq i, j} b_{l \rightarrow j} \right) s_j \right)\end{aligned}$$

and approximated site:

$$\begin{aligned}\tilde{p}(s_i, s_j) &\propto \tilde{g}_{ij}(s_i, s_j) \tilde{f}_i(s_i) \left(\sum_{s_k} \prod_{k \in ne(i) \setminus j} \tilde{g}_{ki}(s_k, s_i) \right) \tilde{f}_j(s_j) \left(\sum_{s_l} \prod_{l \in ne(j) \setminus i} \tilde{g}_{lj}(s_l, s_j) \right) \\ &\propto \exp(b_{j \rightarrow i} s_i) \exp(b_{i \rightarrow j} s_j) \exp(a_i s_i) \exp \left(\sum_{k \neq i, j} b_{k \rightarrow i} s_i \right) \exp(a_j s_j) \exp \left(\sum_{l \neq i, j} b_{l \rightarrow j} s_j \right) \\ &= \exp \left(\left(b_{j \rightarrow i} + a_i + \sum_{k \neq i, j} b_{k \rightarrow i} \right) s_i \right) \exp \left(\left(b_{i \rightarrow j} + a_j + \sum_{l \neq i, j} b_{l \rightarrow j} \right) s_j \right)\end{aligned}$$

We do moment matching of sufficient statistics (in this case $s_i s_j$):

$$\langle s_i \rangle_{\tilde{p}_{s_i}} = \langle s_i \rangle_{\hat{p}(s_i, s_j)}$$

$$\langle s_j \rangle_{\tilde{p}_{s_j}} = \langle s_j \rangle_{\hat{p}(s_i, s_j)}$$

We denote:

$$\theta_i = a_i + \sum_{k \neq i, j} b_{k \rightarrow i},$$

and

$$\theta_j = a_j + \sum_{l \neq i, j} b_{l \rightarrow j}.$$

Thus we have, by matching marginal expectation:

$$\frac{\exp(b_{j \rightarrow i} + \theta_i)}{\exp(b_{j \rightarrow i} + \theta_i) + 1} = \frac{\exp \left(-\frac{\mu_i^\top \mu_j}{\sigma^2} + \theta_i + \theta_j \right) + \exp(\theta_i)}{\exp \left(-\frac{\mu_i^\top \mu_j}{\sigma^2} + \theta_i + \theta_j \right) + \exp(\theta_i) + \exp(\theta_j) + 1}$$

and we have:

$$b_{j \rightarrow i} = \log \left(\frac{\exp \left(-\frac{\mu_i^\top \mu_j}{\sigma^2} + \theta_j \right) + 1}{\exp(\theta_j) + 1} \right)$$

and

$$b_{i \rightarrow j} = \log \left(\frac{\exp \left(-\frac{\mu_i^\top \mu_j}{\sigma^2} + \theta_i \right) + 1}{\exp(\theta_i) + 1} \right)$$

We omit the explicit normalising constant inside each factor as we only require matching moments and adjust the site parameters $a_i, b_{j \rightarrow i}$. We also did not make any approximation on true global normaliser. We calculate the normaliser implied by the approximated factors. To calculate the expectation of sufficient statistics, we have:

$$q(\mathbf{s}) \propto \prod_i \exp(a_i s_i) \prod_{i < j} \exp(b_{j \rightarrow i} s_i) \exp(b_{i \rightarrow j} s_j)$$

$$\langle s_i \rangle_{q(s)} = q(s_i = 1) = \sigma \left(a_i + \sum_{j \neq i} b_{j \rightarrow i} \right)$$

and

$$\langle s_i s_j \rangle_{q(s)} = \langle s_i \rangle_{q(s)} \langle s_j \rangle_{q(s)} = \sigma \left(a_i + \sum_{j \neq i} b_{j \rightarrow i} \right) \sigma \left(a_j + \sum_{k \neq j} b_{k \rightarrow j} \right)$$

where $\sigma(\cdot)$ is sigmoid function.

We also did not make any assumptions on other parameters which are used to learn a_i and $b_{j \rightarrow i}, b_{i \rightarrow j}$ such as μ, σ, π . We will use EM to learn those parameters.

(c)

If we use factored approximate messages, we have:

$$g_{ij}(s_i, s_j) \approx \tilde{g}_{ij}(s_i, s_j) = M_{i \rightarrow j}(s_j) M_{j \rightarrow i}(s_i)$$

then the full joint is approximated by a factored distribution:

$$p(\mathcal{S}) \approx \frac{1}{Z} \prod_i f_i(s_i) \prod_{i < j} \tilde{g}_{ij}(s_i, s_j) = \frac{1}{Z} \prod_i \left(f_i(s_i) \prod_{j \in ne(i)} M_{j \rightarrow i}(s_i) \right)$$

This is essentially Loopy BP. With this factored approximate messages, we have:

$$\begin{aligned} q_{\neg i, j}(s_i, s_j) &= \tilde{f}_i(s_i) \prod_{k \in ne(i) \setminus j} M_{k \rightarrow i}(s_i) \tilde{f}_j(s_j) \prod_{l \in ne(j) \setminus i} M_{l \rightarrow j}(s_j) = q_{\neg i, j}(s_i) q_{\neg i, j}(s_j) \\ \tilde{g}_{ij}^{\text{new}}(s_i, s_j) &\leftarrow \arg \min_g \mathbf{KL} [g_{ij}(s_i, s_j) q_{\neg i, j}(s_i, s_j) \| g(s_i, s_j) q_{\neg i, j}(s_i, s_j)] \\ \{M_{i \rightarrow j}^{\text{new}}, M_{j \rightarrow i}^{\text{new}}\} &\leftarrow \arg \min \mathbf{KL} [g_{ij}(s_i, s_j) q_{\neg i, j}(s_i, s_j) \| M_{i \rightarrow j}(s_j) M_{j \rightarrow i}(s_i) q_{\neg i, j}(s_i, s_j)] \\ &= \arg \min \mathbf{KL} [g_{ij}(s_i, s_j) q_{\neg i, j}(s_i) q_{\neg i, j}(s_j) \| M_{i \rightarrow j}(s_j) q_{\neg i, j}(s_j) M_{j \rightarrow i}(s_i) q_{\neg i, j}(s_i)] \end{aligned}$$

So we have:

$$\begin{aligned} M_{i \rightarrow j}^{\text{new}}(s_j) &= \sum_{s_i} \left(g_{ij}(s_i, s_j) \tilde{f}_i(s_i) \prod_{k \in ne(i) \setminus j} M_{k \rightarrow i}(s_i) \right) \\ M_{j \rightarrow i}^{\text{new}}(s_i) &= \sum_{s_j} \left(g_{ij}(s_i, s_j) \tilde{f}_j(s_j) \prod_{l \in ne(j) \setminus i} M_{l \rightarrow j}(s_j) \right) \end{aligned}$$

$$\begin{aligned}\tilde{f}_i^{\text{new}}(s_i) &\leftarrow \arg \min_f \mathbf{KL} [f_i(s_i)q_{\neg i}(s_i) \| f(s_i)q_{\neg i}(s_i)] \\ &= \arg \min_f \mathbf{KL} \left[f_i(s_i) \prod_{j \in ne(i)} M_{j \rightarrow i}(s_i) \| f(s_i) \prod_{j \in ne(i)} M_{j \rightarrow i}(s_i) \right]\end{aligned}$$

We make same approximation as (b):

$$\tilde{f}_i(s_i) = \exp(a_i s_i)$$

and

$$\tilde{g}_{ij}(s_i, s_j) = M_{i \rightarrow j}(s_j) M_{j \rightarrow i}(s_i) = \exp(b_{i \rightarrow j} s_j) \exp(b_{j \rightarrow i} s_i)$$

To update a_i , we have tilted marginal distribution:

$$\hat{p}(s_i) \propto f_i(s_i) \prod_{j \in ne(i)} M_{j \rightarrow i}(s_i)$$

and approximated site:

$$\tilde{p}(s_i) \propto \tilde{f}_i(s_i) \prod_{j \in ne(i)} M_{j \rightarrow i}(s_i)$$

We do moment matching of sufficient statistics (in this case s_i):

$$\langle s_i \rangle_{\tilde{p}(s_i)} = \langle s_i \rangle_{\hat{p}(s_i)}$$

To update b_{ij} , we have joint tilted distribution:

$$\hat{p}(s_i, s_j) \propto g_{ij}(s_i, s_j) \tilde{f}_i(s_i) \prod_{k \in ne(i) \setminus j} M_{k \rightarrow i}(s_i) \tilde{f}_j(s_j) \prod_{l \in ne(j) \setminus i} M_{l \rightarrow j}(s_j)$$

and approximated site:

$$\tilde{p}(s_i, s_j) \propto \tilde{f}_i(s_i) \prod_{j \in ne(i)} M_{j \rightarrow i}(s_i) \tilde{f}_j(s_j) \prod_{i \in ne(j)} M_{i \rightarrow j}(s_j) =: \tilde{p}(s_i) \tilde{p}(s_j)$$

We do moment matching of sufficient statistics (in this case $s_i s_j$):

$$\langle s_i \rangle_{\tilde{p}_{s_i}} = \langle s_i \rangle_{\hat{p}(s_i, s_j)}$$

$$\langle s_j \rangle_{\tilde{p}_{s_j}} = \langle s_j \rangle_{\hat{p}(s_i, s_j)}$$

which give us same update as (b).

This is loopy BP algorithm as messages traverse the same edges multiple times due to cycles.

(d)

From (b) the approximation we made, it has same posterior structure as we had in mean field where we assume full factored independent Bernoulli distribution. In Loopy BP, we also have Bernoulli distribution with each λ_{ni} to be:

$$\langle s_i \rangle_{q(s)} = q(s_i = 1) = \sigma \left(a_i + \sum_{j \neq i} b_{j \rightarrow i} \right)$$

This means we can learn loopy BP with other parameters using EM algorithm similar to Q3 (just change E-step). To select K , we can have same treatment as in Q4 where we pose prior beliefs on all parameters and use automatic relevance determination to select K . Please refer to Q4 for details. The only difference is the way of updating λ_{ni} in E-step.

Q6

We can make most use of the code in Q3 and only modify the E-step to include Loopy BP update. Here is the implementation.

Note in loopy BP learning, sometimes $b_{j \rightarrow i}$ fail to stabilise. In this case the free energy will not always increase. We keep track of this case when $b_{j \rightarrow i}$ fail to converge and we select the maximum free energy during the iterations and corresponding $b_{j \rightarrow i}$ and λ_{ni} . This can keep free energy always increasing.

Initialisation is also very important. We initialised $b_{j \rightarrow i}$ and λ_{ni} randomly. To make sure we achieve good results, we run multiple times with different random seeds and select the highest free energy.

We also use damping in updating $b_{j \rightarrow i}$ to avoid over-updating and make it more stable. We set damping parameter to 0.5.

The result of features learnt in 1.22 is similar to 1.16 in Q3. Their free energy levels are also similar. However, loopy BP is not very stable in E-step. We need to take extra effort to make it work properly. Theoretically speaking, loopy BP can assume pairwise factors which are richer than mean field approximation but we simplified loopy BP to remove the cross relation between factors. This makes our loopy BP behave same as mean field approximation but with extra implementation effort. To better utilise the characteristics of loopy BP, we can make less over-simplified approximation to capture more than mean field approximation, but that would require more complicated learning steps.

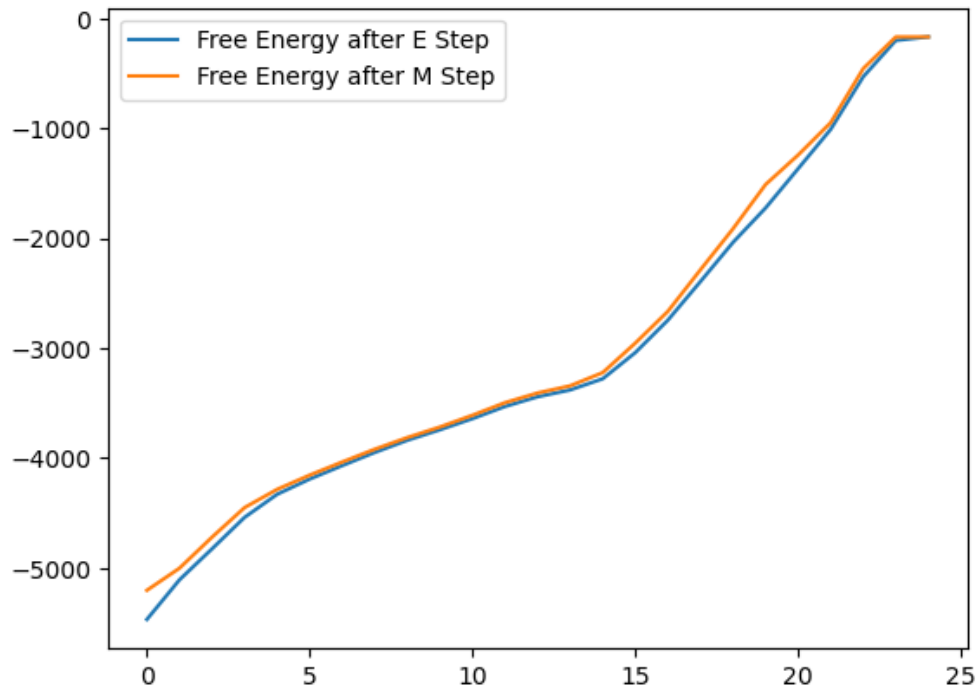


Figure 1.21. EM Free Energy for Loopy BP

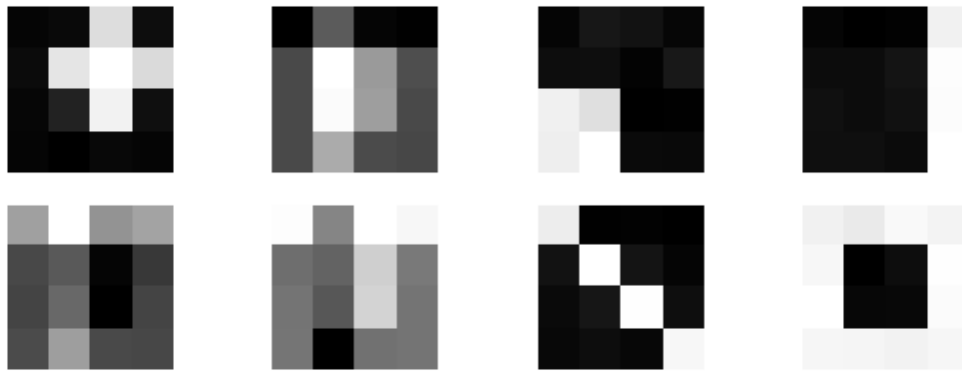


Figure 1.22. Features learnt in Loopy BP EM

```

1 class LoopyBP_EM:
2
3     def __init__(self, data, K, random_seed=1994):
4         """
5         data: (N, D) np array, data
6         K : Number of hidden binary variables
7         """
8         np.random.seed(random_seed)
9         self.X = data.copy()

```

```

10     self.K = K
11     self.N, self.D = data.shape
12     self.a = np.zeros((self.N, self.K))
13     self.b = np.random.rand(self.N, self.K, self.K)
14     # fill self.b with 0 on the diagonal
15     for n in range(self.N):
16         np.fill_diagonal(self.b[n], 0.0)
17     self.lmbda = np.random.rand(self.N, self.K)
18     # We run M step before the full EM algorithm
19     # These will be initialised in M step
20     self.sigma = None
21     self.mu = None
22     self.pie = None
23
24
25     def LoopyBP(self, maxsteps, tol, damping=0.5):
26         """
27         E step doing loopy BP
28
29         Input:
30         mu:          (D, K) np array, means
31         sigma:       scalar, variance of data
32         pie:         (K,) np array, prior on s
33         maxsteps:    max iteration
34         tol:         tolerance of halting the iteration
35         damping:     damping factor for updating b
36
37         Output:
38         Fs:          Free energy in each iteration
39         """
40         K = self.K
41         mu2 = self.mu.T @ self.mu # (K, K)
42         mu2_diag = np.diag(mu2)
43         mu2_offdiag = mu2 - np.diag(mu2_diag)
44         mu_x = self.X @ self.mu # (N, K)
45         log_pi = np.log(self.pie.flatten()) # (K, )
46         log_one_minus_pi = np.log(1 - self.pie.flatten()) # (K, )
47         log_pi_one_minus_pi = log_pi - log_one_minus_pi # (K, )
48         lmbda_mu_cross = np.array([lmbda_row @ mu2_offdiag @
49                                     lmbda_row.T for lmbda_row in self.lmbda])
50         F_n = (
51             np.sum(self.lmbda * log_pi_one_minus_pi, axis=1) + np.
52                 sum(log_one_minus_pi) # Prior term
53             - np.sum(self.lmbda * np.log(self.lmbda) + (1 - self.
54                 lmbda) * np.log(1 - self.lmbda), axis=1) # Entropy
55                 term
56             - 0.5 / self.sigma**2 * (np.sum(self.X**2, axis=1) - 2 *

```

```

        np.sum(self.lmbda * mu_x, axis=1) + lambda_mu_cross.
        flatten() + np.sum(self.lmbda * mu2_diag[None, :],
        axis=1)) # Likelihood term
53     - self.D / 2 * np.log(2 * np.pi * self.sigma**2) #
        Normalisation term
54 )
55 F = np.sum(F_n)
56 Fs = [F]
57 self.a = mu_x / self.sigma**2 - 0.5 * mu2_diag[None, :] /
        self.sigma**2 + log_pi_one_minus_pi[None, :]
58 old_b = self.b.copy()
59 old_lambda = self.lmbda.copy()
60 increasing_idx = 0
61 max_F = F
62 for _ in range(maxsteps):
63     b_new = self.b.copy()
64     for j in range(K):
65         for i in range(j + 1, K):
66             # Update b_{j -> i} using theta_j
67             theta_j = self.a[:, j] + np.sum(self.b[:, :, j],
        axis=1) - self.b[:, j, j] - self.b[:, i, j]
68             numerator_j = np.exp(-mu2[i, j] / self.sigma**2
        + theta_j) + 1
69             denominator_j = np.exp(theta_j) + 1
70             b_update_j = np.log(numerator_j / denominator_j)
71
72             # Update b_{i -> j} using theta_i
73             theta_i = self.a[:, i] + np.sum(self.b[:, :, i],
        axis=1) - self.b[:, i, i] - self.b[:, j, i]
74             numerator_i = np.exp(-mu2[i, j] / self.sigma**2
        + theta_i) + 1
75             denominator_i = np.exp(theta_i) + 1
76             b_update_i = np.log(numerator_i / denominator_i)
77
78             # Apply damping
79             b_new[:, j, i] = damping * b_update_j + (1 -
        damping) * self.b[:, j, i]
80             b_new[:, i, j] = damping * b_update_i + (1 -
        damping) * self.b[:, i, j]
81         diff = np.max(np.abs(self.b - b_new))
82         self.b = b_new
83         logit_1 = self.a + np.sum(self.b, axis=1) - np.diagonal(
        self.b, axis1=1, axis2=2)
84         self.lmbda = np.clip(1.0 / (1.0 + np.exp(-logit_1)), 1e
        -8, 1 - 1e-8)
85
86         lambda_mu_cross = np.array([lambda_row @ mu2_offdiag @

```

```

        lambda_row.T for lambda_row in self.lambda])

87
88     F_n = (
89         np.sum(self.lambda * log_pi_one_minus_pi, axis=1) +
90         np.sum(log_one_minus_pi) # Prior term
91         - np.sum(self.lambda * np.log(self.lambda) + (1 - self
92           .lambda) * np.log(1 - self.lambda), axis=1) #
          Entropy term
93         - 0.5 / self.sigma**2 * (np.sum(self.X**2, axis=1)
94           -2 * np.sum(self.lambda * mu_x, axis=1) +
95           lambda_mu_cross.flatten() + np.sum(self.lambda *
96           mu2_diag[None,:], axis=1)) # Likelihood term
97         - self.D / 2 * np.log(2 * np.pi * self.sigma**2) #
98           Normalisation term
99     )
100
101     F = np.sum(F_n)
102     Fs.append(F)
103     if F > max_F:
104         max_F = F
105         old_b = self.b.copy()
106         old_lambda = self.lambda.copy()
107         increasing_idx = _ + 1
108     if diff < tol:
109         break
110 if Fs[-1] < Fs[0] and diff >= tol:
111     self.b = old_b
112     self.lambda = old_lambda
113     Fs = Fs[:increasing_idx + 1]
114 return Fs
115
116 def m_step(self):
117     """
118     We calculate the M step of the EM algorithm:
119     -----
120     ES: shape (N, K) E_q[s]
121     ESS: shape (K, K) sum over data points of E_q[ss'] (N, K
122           , K)
123
124           if E_q[ss'] is provided, the sum over N
125           is done for you.
126
127     Update these parameters:
128     -----
129     mu: shape (D, K) matrix of means in p(y|{s_i},mu,sigma)
130     sigma: shape (,) standard deviation in same
131     pie: shape (1, K) vector of parameters specifying
132           generative distribution for s

```

Output:

```

F: scalar, the free energy
"""
N = self.N
ES = self.lmbda
ESS = np.array([this_lmbda.reshape(-1, 1) * this_lmbda.
                 reshape(1, -1) for this_lmbda in self.lmbda])
for n in range(N):
    np.fill_diagonal(ESS[n, :, :], self.lmbda[n])
K = self.K
ESS = np.sum(ESS, axis=0)
if ESS.shape != (K, K):
    raise TypeError('ESS must be square and have the same
                    number of columns as ES')

self.mu = np.dot(np.dot(np.linalg.inv(ESS), ES.T), self.X).T
self.sigma = np.sqrt((np.trace(np.dot(self.X.T, self.X)) +
                      np.trace(np.dot(np.dot(self.mu.T, self.mu), ESS))
                      - 2 * np.trace(np.dot(np.dot(ES.T, self.X),
                                                self.mu))) / (N * D))

self.pie = np.mean(ES, axis=0, keepdims=True)
self.pie = np.clip(self.pie, 1e-8, 1 - 1e-8)

mu2 = self.mu.T @ self.mu # (K, K)
mu2_diag = np.diag(mu2)
mu2_offdiag = mu2 - np.diag(mu2_diag)
mu_x = self.X @ self.mu # (N, K)
log_pi = np.log(self.pie.flatten()) # (K, )
log_one_minus_pi = np.log(1 - self.pie.flatten()) # (K, )
log_pi_one_minus_pi = log_pi - log_one_minus_pi # (K, )
lmbda_mu_cross = np.array([lmbda_row @ mu2_offdiag @
                           lmbda_row.T for lmbda_row in self.lmbda])
F_n = (
    np.sum(self.lmbda * log_pi_one_minus_pi, axis=1) +
    np.sum(log_one_minus_pi) # Prior term
    - np.sum(self.lmbda * np.log(self.lmbda) + (1 - self
        .lmbda) * np.log(1 - self.lmbda), axis=1) #
        Entropy term
    - 0.5 / self.sigma**2 * (np.sum(self.X**2, axis=1)
        -2 * np.sum(self.lmbda * mu_x, axis=1) +
        lmbda_mu_cross.flatten() + np.sum(self.lmbda *
        mu2_diag[None,:], axis=1)) # Likelihood term
    - self.D / 2 * np.log(2 * np.pi * self.sigma**2) #
        Normalisation term
)
```

```

158         F = np.sum(F_n)
159         return F
160
161
162     def run(self, max_iter, max_E_iter, tol, damping=0.5):
163         E_Fs = []
164         M_Fs = []
165         # run M step once to initialise mu, pie and sigma
166         self.m_step()
167         for _ in range(max_iter):
168             # E step
169             E_F = self.LoopyBP(max_E_iter, 1.0e-8, damping)
170             E_Fs.append(E_F[-1])
171             # M step
172             M_F = self.m_step()
173             M_Fs.append(M_F)
174             # check convergence
175             if len(M_Fs) > 1 and np.abs(M_Fs[-1] - M_Fs[-2]) < tol:
176                 break
177         return E_Fs, M_Fs

```

Listing 1.12. Loopy BP

```

1 K = 8
2 random_seeds = [123, 234, 672, 2987, 9271, 18888, 20000, 439, 219,
3                 1994, 2000, 2025]
4 E_Fs = []
5 M_Fs = []
6 for i, rs in enumerate(random_seeds):
7     loopyEP = LoopyBP_EM(Y, K, rs)
8     this_E_Fs, this_M_Fs = loopyEP.run(100, 50, 1.0e-8, 0.5)
9     E_Fs.append(this_E_Fs[-1])
10    M_Fs.append(this_M_Fs[-1])
11 i_max = np.argmax(M_Fs)
12 loopyEP = LoopyBP_EM(Y, K, random_seeds[i_max])
13 E_Fs, M_Fs = loopyEP.run(100, 50, 1.0e-8, 0.5)

```

Listing 1.13. Loopy BP Random Seed Selection